

The impact of artificial intelligence design on pricing

John Asker^{1,2}  | Chaim Fershtman^{3,4} | Ariel Pakes^{2,5}

¹Department of Economics, UCLA, Los Angeles, California, USA

²National Bureau of Economic Research (NBER), Cambridge, Massachusetts, USA

³The Eitan Berglas School of Economics, Tel Aviv University, Tel Aviv, Israel

⁴Centre for Economic Policy Research (CEPR), London, UK

⁵Department of Economics, Harvard University, Cambridge, Massachusetts, USA

Correspondence

John Asker, Department of Economics, UCLA, Los Angeles, California, USA; National Bureau of Economic Research (NBER), Cambridge, Massachusetts, USA.
Email: johnasker@gmail.com and johnasker@econ.ucla.edu

Abstract

The behavior of artificial intelligence (AI) algorithms is shaped by how they learn about their environment. We compare the prices generated by AIs that use different learning protocols when there is market interaction. Asynchronous learning occurs when the AI only learns about the return from the action it took. Synchronous learning occurs when the AI conducts counterfactuals to learn about the returns it would have earned had it taken an alternative action. The two lead to markedly different market prices. When future profits are not given positive weight by the AI, (perfect) synchronous updating leads to competitive pricing, while asynchronous can lead to pricing close to monopoly levels. We investigate how this result varies when either counterfactuals can only be calculated imperfectly and/or when the AI places a weight on future profits. Lastly, we investigate performance differences between offline and online play.

1 | INTRODUCTION

Firms increasingly delegate their pricing to algorithms that exploit detailed data on customers' preferences and, in some instances, use a learning process to develop strategies to play an oligopolistic pricing game in the presence of competitors.¹ Artificial intelligence algorithms (AIs) provide a prominent approach to implementation. AIs operate by learning about the returns to taking feasible actions, and then performing the action that they have learned works best. The embedded learning process is at the heart of how an AI performs. This paper shows that when pricing decisions are delegated to AIs the way in which AIs learn can have an economically significant impact on the pricing outcomes realized in the market.

This paper focuses on reinforcement learning. Reinforcement learning is a common way to implement an AI.² In reinforcement learning each action at each state is given a value. Learning occurs by updating these values from the information gathered by the AI during the course of play. The optimal action at each state is the one with the highest value.³

For clarity, our initial focus is on two learning protocols that lie at opposite extremes in terms of the information that they leverage. Consider a market interaction in which Firm *A* charges \$7 and its rival, *B*, charges \$8. The first learning approach we investigate, asynchronous learning, allows the AI to learn only from actions that are actually taken. So, in the two firm market above, all the AI can learn from is the profit that it realizes from charging \$7.⁴ The second learning approach we investigate, perfect synchronous learning, allows the AI to conduct counterfactuals to assist learning. When an AI uses perfect synchronous learning, it observes its own profit and that the rival charged \$8, and it can construct the counterfactual profit that would have arisen had it chosen \$6, or any other feasible price.

Synchronous learning requires some understanding of the underlying economic environment.⁵ In the implementation discussed above, constructing accurate counterfactuals requires an understanding of demand

conditions, competitors' prices, and the market clearing rule. Asynchronous learning requires no understanding of any of this. An AI using asynchronous learning needs no information other than the action it took and the profit it realized.

The influence of these different learning protocols on pricing outcomes can be seen in Figure 1. Figure 1 shows prices in a market in which two AIs sell products that are perfect substitutes in Bertrand competition. The AIs do not care about future profits. Marginal costs are constant and equal to 2. Nash equilibrium prices are just above marginal costs (just above due to the discreteness in the set of feasible prices). The monopoly price is 10. When both AIs employ perfect synchronous learning, they converge to Nash pricing quickly. By contrast, when both AIs employ asynchronous learning they converge to prices that are substantially above marginal costs.

This paper endeavors to explain why this difference in outcomes occurs and evaluates the extent to which it is a general feature that is robust across a variety of environments. In doing so it seeks to contribute to the growing discussion in the economics, legal, and policy literature on the impact of the adoption of AI pricing algorithms on the competitiveness of markets.⁶

We focus on the two extreme cases of asynchronous and perfect synchronous learning to make the pricing implications of learning protocols from AI pricing algorithms transparent. Much of the academic literature on AI pricing algorithms uses asynchronous updating.⁷ This seems at odds with how we would expect an AI pricing algorithm to operate in a real market setting as it ignores information which likely would be viewed as helpful.

Even a simple shopkeeper who sees the demand generated at a given price realizes that this has implications for the demand that would be realized at alternative prices. The process of inferring what demand would have been at alternative prices is a synchronous updating process. The precision of the shopkeeper's estimates of demand at alternative prices would depend on the information available on the underlying demand system and on whether competitors' prices are observed. However, even an understanding that the demand function is downward sloping informs perceptions about the values of taking different actions and, as we confirm in Section 5.1, can have dramatic effects on realized prices. We expect that a sophisticated AI pricing algorithm would "understand" that demand slopes downward as well. As a result, the view we take in this paper is that actual AI pricing algorithms live on a spectrum with perfect synchronous and asynchronous algorithms at either end. The position of any algorithm on the spectrum is likely to be determined by the willingness of the firm to invest in algorithmic design, the access the algorithm has to data as well as the periodicity at which data appears, and the ability the algorithm provides managers to chose alternative algorithmic settings.

The basic model, a simple Bertrand setting, with two identical firms selling identical goods, is briefly described in Section 2. A more formal description of the mechanics of the reinforcement learning algorithm that we investigate makes up the rest of that section. A core result, expanding on Figure 1, is contained in Section 3. There the full set of computational results for this base case is provided. These results mirror those in Figure 1—when the future has no value (the discount factor, $\beta = 0$); that is, asynchronous updating leads to supra-competitive pricing while perfect synchronous updating does not. The computational results delve deeper into why this is the case.⁸

The asynchronous result is a function of how the algorithm updates the value of the action taken. In a market with competing AI pricing algorithms, both will choose to play the same action in a period if that action is perceived to have the highest value by both algorithms. When subsequent updating does not decrease the value attached to this action, or

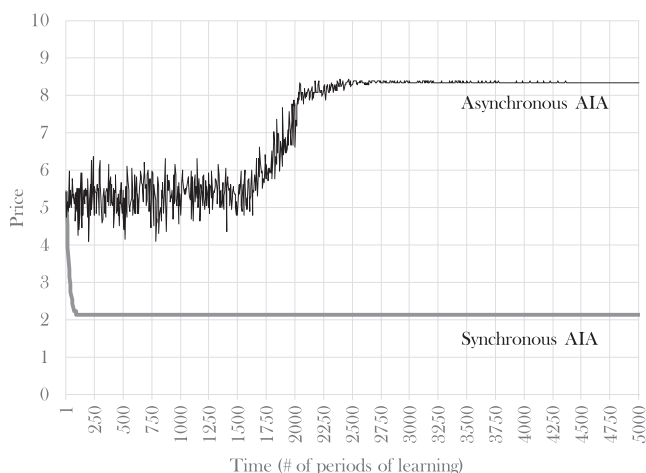


FIGURE 1 Price paths with different algorithm designs. The median price (vertical axis) across 100 simulated runs by period (horizontal axis) is shown for firm 1, in a static Bertrand market in which two firms are selling homogeneous goods. The model is parametrized as follows. Market demand is $Q = 1$ if $p \leq 10$, zero otherwise. Marginal cost = 2. There are 100 feasible prices equally spaced between 0.1 and 10 inclusive. Firms put zero weight on future profits. The model is parametrized as per Figure 2. See notes therein for further details.

increase the value attached to other actions, the current action will become a “rest point” and the AIs will have converged to a set of stable prices. Computational results show that this process leads to supra-competitive pricing.

We then provide a proposition with two parts, one of which formalizes the asynchronous computational results, and the other shows that perfect synchronous updating, absent any value being placed on the future (i.e., $\beta = 0$) leads to competitive (Nash) pricing. The proposition requires only mild regularity conditions; in particular it allows for history-dependent pricing, different demand systems, and so on, provided the algorithm is programed to maximize current profits. So we view the computational results as robust for the $\beta = 0$ case. Indeed they provide a way of computing Nash equilibrium prices anytime they are needed.

We then delve deeper into the behavior underlying the rest points reached by perfect synchronous and asynchronous updating by showing that each satisfies one of the two sets of equilibrium conditions used in the experienced-based equilibrium (EBE) paper of Fershtman and Pakes (2012). Asynchronous updating finds rest points that are EBE, while perfect synchronous updating finds rest-points that satisfy the restricted experienced-based equilibrium (REBE) concept. This result does not require $\beta = 0$ or any restriction on the profit function or the state space (so it allows for history-dependent strategies). It therefore provides a basis for understanding the rest points generated by the broader range of algorithms we turn to next.

The base case is extended in a variety of ways in Section 5. Section 5.1 extends the $\beta = 0$ case to allow for increases in the number of firms, experimentation, and intermediate cases lying between the extremes of perfect synchronous and asynchronous updating.⁹ Section 5.2 first uses the equilibrium concepts introduced earlier to explain why the results for the $\beta > 0$ case can be different, and then considers the extensions listed above for that case.

As would be expected, having more firms in a market mitigates the price inflation induced by asynchronous updating. In the static setting ($\beta = 0$), a market with five firms displays little propensity to settle on a price above competitive levels.

Experimentation means that the AI periodically chooses a suboptimal action to explore its payoff.¹⁰ At first glance this experimentation may seem like a substitute for the informationally demanding process of perfect synchronous updating. Since we cannot discuss rest points if experimentation continues indefinitely, we provide a series of examples in which experimentation is allowed to occur for twice the length of time that the base case asynchronous algorithm takes to converge. Experimentation is then stopped and the algorithm is continued until it reaches a rest point.

What we find is that experimentation does not fully mitigate the price-inflating propensity of asynchronous updating. In fact, experimentation has only limited success at reducing prices, and it comes at a significant cost in computational burden.¹¹ The limited success and additional computational burden imposed by experimentation, coupled with the potential financial cost of implementing random prices, leads us to be skeptical that naive experimentation is a practical solution to mitigating the price elevation introduced by asynchronous algorithms.

Given the ineffectiveness of experimentation in mitigating price elevation, we consider augmenting the basic asynchronous updating procedure with a simple insight from economics: that demand curves slope downward (creating a low-information form of synchronous updating). The knowledge that this is the case allows the algorithm to use an easily computable bound to update the value the algorithm assigns to counterfactual prices which are inconsistent with the observed result and the assumption that demand decreases in price. Leveraging this additional structure is far more effective at mitigating the supra-competitive pricing of asynchronous updating than experimentation. It is also less costly and easier to compute.

In Section 5.2 we consider settings in which the future is given positive weight by the algorithm (what we call the “repeated” setting, with $\beta > 0$). This is done in models in which the state space includes the prices chosen in the prior period; so policies are history dependent. As in prior work (notably, Calvano et al., 2020), asynchronous updating leads to supra-competitive pricing (indeed, often replicating monopoly). As we explain, when $\beta > 0$ perfect synchronous updating can also lead to supra-competitive pricing. We find that it does, albeit to significantly lower prices than in the asynchronous case.

This extends the results in Calvano et al. (2020) to cases where synchronous updating is used. We show analytically that one source of price elevation in the synchronous case is analogous to the incentives appearing in trigger price equilibrium; that is, the “threat” of lower cashflows in the future in the event of “defection.” However, a comparison to models with no history-dependent pricing, that is, models which do not allow the algorithm to mimic deviation cum punishment schemes to support collusion, makes it clear that a significant amount of the difference between the prices at the rest point and Nash equilibrium prices has nothing to do with such punishment schemes.

Our pricing results, and those of almost all of the papers that deal with competition between AI pricing algorithms, focus on the properties of converged policies from games simulated on computers. As we show, these simulations

frequently require a large number of iterations before convergence. Though it is easy to imagine that a firm may use computer simulations to explore alternative policies and “train” an algorithm “offline,” the exploration process can be costly when done “online,” that is, while competing for profits in an actual market setting. Then exploration requires forces the agent to use a price which the algorithm currently perceives to be in-optimal. Section of the paper considers the likely differences between online and offline play, and concludes with a word of caution in interpreting the converged policies from offline play as an adequate approximation to online play.

1.1 | Related literature

Our paper is related to several strands of literature. A number of papers investigate how AIs that employ reinforcement learning could lead to pricing outcomes that are collusive in nature (see, for instance, Calvano et al., 2020, 2021a; den Boer et al., 2022; Epivent & Lambin, 2022; Eschenbaum et al., 2022; Klein, 2019).¹² Calvano et al. consider a differentiated product pricing game with logit demands. We extend this work by examining how variation in the structure of the AI varies the degree to which prices deviate from the static Nash behavior.¹³

A related line of research considers settings in which the adoption of algorithms in the pricing process allows firms to better infer demand and how this may impact the propensity to collude. Miklós-Thal and Tucker (2019) construct a theoretical model in which algorithms enhance the ability of firms to forecast demand. They argue that this can lead firms to deviate from collusive conduct more frequently, leading to lower prices and higher consumer surplus (for a similar argument see also O'Connor & Wilson, 2019; Martin & Rasch, 2022). Hansen et al. (2020) consider a setting in which price experimentation may make demand appear more inelastic than it actually is which leads to supra-competitive pricing. Brown and MacKay (2021) present compelling empirical evidence that the adoption of pricing algorithms materially impacts pricing patterns and complement this with a theoretical exploration of how delegation to algorithms may allow firms to commit to strategies leading to higher prices than would otherwise be the case.¹⁴ Salcedo (2015) considers a somewhat similar theoretical setting, arguing that demand fluctuations can allow firms to infer the algorithms used by competitors and tailor their algorithms to better realize supra-competitive pricing. Leisten (2021) considers competition when humans are able to limit, or override, the decisions of algorithms, and provides conditions under which algorithmic competition, even with active human oversight, can still lead to supra-competitive outcomes.

Our companion paper, Asker et al. (2022), provides a five-page introduction to some of the results discussed in depth in this paper. Its sole focus is on a subset of the the two-player static results. It omits all discussions of equilibrium, formal proofs relating to convergence, experimentation, cases with more than two players, and anything related to dynamic models ($\beta > 0$) or policy. It is best regarded as a terse summary of a subset of the results that this paper discusses in depth.

The effect of the learning procedure on the equilibrium of the game has been discussed in several theoretical papers. Milgrom and Roberts (1991), for example, considered the effect of two types of learning; “Adaptive” and “Sophisticated” on the equilibrium of normal-form games. The learning protocols in Milgrom and Roberts are very different from that considered in this paper. Milgrom and Roberts restrict adaptive learning to be procedures in which the player eventually choose strategies that are nearly best reply to some distribution over its competitors' joint strategies.¹⁵ Note that in our setting the asynchronized updating algorithm does not converge to the Nash equilibrium. Similarly, the synchronized updating that we study is different from the sophisticated updating studied in Milgrom and Roberts. In the sophisticated updating the player uses information that he may possess about his competitors' profit function, information, alternatives, and so on. In our synchronized updating the player does not use such an information and only update his value function for counterfactuals. This distinction may suggest that one can consider an algorithm that is more sophisticated than our synchronized updating using different types of information regarding the competitors and the game that is being played.¹⁶

Empirical evidence as to the impact of the adoption of AI algorithms is lagging. A notable paper in this regard is Assad et al. (2020), which provides an empirical example in which the inferred adoption of algorithmic approaches to pricing by German gasoline retailers appears to have coincided with an increase in margins of up to 38%.

An energetic legal and policy debate over the collusive potential of AIs underlies much of the interest in the academic literature. Harrington (2018) provides an excellent entry-level overview, while also expressing skepticism about the ability of current cartel law to address any tendency for AIs to induce supra-competitive pricing.¹⁷

This paper also connects with a range of other literature in economics. Reinforcement learning, and its relationship to Nash equilibrium, has been investigated in both theoretical (see Fudenberg & Levine, 2016, for a survey) and

experimental (see Erev & Haruvy, 2016, for a survey) literatures. On the theoretical side, that the AIs we investigate do not always converge to Nash pricing is not surprising given Hart and Mas-Colell (2003)'s results.

Lastly, reinforcement learning has a long history as a tool in the computation of dynamic games (see Pakes & McGuire, 2001 for an early implementation in a rich oligopoly model). Fershtman and Pakes (2012) propose the experience-based equilibrium concept which we come back to below and is extended in Asker et al. (2020).

2 | MODEL

This section introduces AI pricing algorithms highlighting a choice that must be made in implementation. To do so in as transparent a context as possible, we focus on results for a simple model of a market—the homogenous good Bertrand duopoly. We begin by describing this market environment, and then explain the different ingredients of the AI algorithm that determine its pricing policies. The next section considers generalizations and a more formal result.

2.1 | The Bertrand pricing game

Consider two firms $i \in \{1, 2\}$ with equal marginal costs: $c_1 = c_2 = c$. Firms are Bertrand competitors selling homogenous goods. We discretize the action space and take the set of possible prices to be $\mathcal{P} = \{p^1, \dots, p^M\}$, with $p^{m+1} - p^m = \xi$, so the increment from one price to the next is constant. The prices of firm i are denoted $p_i \in \mathcal{P}$. We let $D(p)$ be the market demand function. We assume that both firms have the capacity to serve all the demand they face. We assume that consumers buy from the firm with the lowest price. In the case of a tie, firms split demand equally. Thus the demand faced by firm i is

$$d_i(p_i, p_j) = \begin{cases} D(p_i), & \text{if } p_i < p_j, \\ \frac{D(p_i)}{2}, & \text{if } p_j = p_i, \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

For simplicity we assume that there is a price $V > 0$ above which the demand is zero and we assume that $p^M = V$. We denote firm i 's profits as $\pi_i(p_i, p_j)$. In the computational example, we make the assumption that the same quantity is demanded $\forall p < V$. This simplifies implementation.¹⁸ In this case, if $c \notin \mathcal{P}$ then there are two Nash equilibria for this (static) Bertrand game.¹⁹ The first one is $p_i = p_j = p^c$, which is the lowest regular price greater than or equal to marginal cost, c . The second one is $p_i = p_j = p^{c+1}$. Where $p^{c+1} = p^c + \xi$. If $c \in \mathcal{P}$ then there are three equilibria, all symmetric, corresponding to both firms pricing at either c , $c + \xi$ or $c + 2\xi$.²⁰

We allow this stage game to be repeated many times. This repetition allows the AIs, which determine firms' pricing policies, to learn to play. Ultimately we are interested in the pricing outcomes that the AIs converge on. Given this, we are agnostic as to whether the learning occurs offline, through simulated play, or online, through learning from actual interactions with other AIs (or any combination of the two). We now turn to a more detailed description of the AIs.

2.2 | The design of the AI algorithm

The AI algorithm we use to play the Bertrand pricing game is a reinforcement learning algorithm.²¹ In a reinforcement learning algorithm the coder defines a goal for the algorithm (e.g., maximizing profits), sets a procedure for determining initial values for all feasible actions, and formulates a rule which is used to update those values. Whenever it is called on to act, the algorithm chooses the highest valued action (the “greedy” action) with some probability (which may be one) and experiments with other actions otherwise. Having acted, it observes the outcome and updates the values it attaches to actions with an updating rule that the algorithm specifies ex ante. It is this updating that allows experience to inform policies; that is, the updating rule determines how the algorithm learns. Reinforcement learning algorithms are used for a wide range of problems in industry (see, for examples, Sutton & Barto, 2018), and have been used in the economics literature to study the interactions of firms in a market for some time.²²

We will take the goal of the algorithm to be to maximize the expected value of the discounted flow of profits the firm earns. For simplicity our baseline will set the discount factor (β) to zero, resulting in algorithms seeking to maximize present profits, but we will also consider cases where $\beta > 0$, so the discussion below applies to the latter case as well.

A reinforcement learning algorithm has the following ingredients.

- (i) Each firm has a set $\mathcal{S}_i$ which is a set of states for firm i (possibly a singleton). The elements of $\mathcal{S}_i$, that is, the $s_i \in \mathcal{S}_i$, are the components of the firm's information set that the firm conditions on when it determines which action to take. We assume that $\mathcal{S}_i$ has a finite number of elements.²³
- (ii) A set of numbers (or values) for every firm which can be interpreted as the firm's perception of the expected discounted values of net cash-flows conditional on its state and action. That is, for every firm i

$$\mathcal{W}_i = \{W_i(p|s_i)\}_{p \in \mathcal{P}, s_i \in \mathcal{S}_i}.$$

- (iii) A method for choosing an action (in our case, price) in every iteration conditional on $\mathcal{W}_i$.
- (iv) An updating rule: The updating rule uses the information the firm observed during that period to update the $\mathcal{W}_i$'s. That is, it is a function that maps $\mathcal{W}_i^k$ into $\mathcal{W}_i^{k+1}$ based on the information observed at iteration (or period) k .

In our context, a “firm” in the above description is an algorithm. We will be primarily concerned with how the properties of algorithms that converge to a rest point differ with the updating rule built into the algorithm, and, to the extent that it helps understanding, the relationship of those rest points to notions of equilibrium. To be clear about what we mean by this statement, we introduce the following definitions.

Definition 1 (Convergence). We say that the AI pricing algorithm has converged at iteration k^* if policies at all $k \geq k^*$ are the same as policies at k^* for all $s \in S$.

Definition 2 (Rest Point). We will say that the AI pricing algorithm reaches a rest point by iteration $k = k^*$ if for all $k \geq k^*$ prices are constant, that is, $\{(p_i^k, p_j^k) = (p_i, p_j)\}_{k \geq k^*}$.

An AI algorithm can converge to a set of strategies which do not satisfy the rest point condition. For example the converged process might put positive probability on more than one price vector, as in the Edgeworth cycle example studied in Maskin and Tirole (1988). On the other hand, if the pricing algorithm has reached a rest point the Markov process underlying it has converged.

When designing a reinforcement learning algorithm, certain choices need to be made. As will be shown in the rest of the paper, these choices can have a material impact on the observed price outcomes. As a result, some comments on each of the components of the algorithm will prove useful.

The state space $\mathcal{S}_i$: In a reinforcement learning algorithm the state space defines what the algorithm knows about its environment, and hence what it can condition its actions on. If the state space is a singleton, then from the point of view of the algorithm every period is the same as every other. If the state space includes some information about the history of past play, then it becomes possible for the algorithm to condition actions on that history (although the extent to which it does so will depend on details of the algorithm). Similarly, if the environment is changing (say, demand shifts) and this change is registered in the state space, then the algorithm can condition actions on that information.²⁴

Firms in a given market may condition on different state spaces. If they do, then $\mathcal{S}_i \neq \mathcal{S}_j$, and if these are the only two firms then the market's states are given by $S = (\mathcal{S}_i, \mathcal{S}_j)$. For example, firms may condition their prices on different functions of what they observed in the past,²⁵ and not all variables observed by one firm need be observed by other firms.

The values $\mathcal{W}_i^k = \{W_i^k(p|s_i)\}$: Each possible action at each state has a value attached to it in each period (or iteration), our k . The designer of the algorithm determines the procedure for choosing the initial values assigned to each element of $\mathcal{W}_i$, say $\mathcal{W}_i^0 = \{W_i^0(p|s_i)\}_{p \in \mathcal{P}, s_i \in \mathcal{S}_i}$.

The choice of action: In the absence of any experimentation by the reinforcement learning algorithm, the algorithm chooses the action with the highest $W_i^k(p|s_i)$ at the state (the s_i) it finds itself in. That is it pursues a “greedy” policy which we denote by $p^{k,*}$, where $p^{k,*} \in \arg \max \{W_i^k(p|s_i)\}$. When experimentation is possible we consider variants on “ ϵ –greedy” policies. That is, the greedy policy is pursued with probability $1 - \epsilon^k$, and with probability ϵ^k a policy is selected randomly. The k indexes the ϵ^k to allow it to vary with the iteration of the algorithm.

Note that there is a sense that experimentation is costly, as the player does not use the action it perceives to be optimal at the time of play. This loss must be set against the increment in future profits generated by exploring what the returns might be from other actions: the familiar exploitation versus experimentation tradeoff which we return to below.

The updating rule: After every period k the algorithm updates one or more of the values $W_i^k(p|s_i)$ that are in memory. That is, for at least some combinations of p and s_i , $W_i^k(p|s_i)$ gets updated and transitions to $W_i^{k+1}(p|s_i)$. If a $W_i^k(p|s_i)$ is not updated, $W_i^{k+1}(p|s_i) = W_i^k(p|s_i)$. There are different possible updating rules, and the rules that are feasible are limited by both the information the algorithm can access on competitor's play and by what market-specific knowledge the algorithm has built into it.

It is useful to begin by distinguishing between two extremes of possible updating rules.

1. *Asynchronous updating:* If the state in the current period is s_i , then only $W_i^k(p_i^k|s_i)$ is updated, where p_i^k is the action (price) chosen by firm i in period k .²⁶
2. *Synchronous updating:* If the state in the current period is s_i , then $W_i^k(p|s_i)$ is update for all $p \in P$ at that s_i .

We now detail the updating rules we use in this paper. Let s_i^k be the state of firm i in period k and p_i^k (p_j^k) be the price chosen by firm i (j). To make our points in a transparent way, most of the text assumes that the transition from the current state (s_i^k) to the state in the next period (s_i^{k+1}) is deterministic (later we return to the complications that arise if experimentation is built into the algorithm). If $W_i^k(p|s_i)$ is changed during the update

$$W_i^{k+1}(p|s_i^k) = \alpha(k) \left[\pi_i(s_i^k, p) + \beta \left(\max_{y \in P} \left\{ W_i^k(y|s_i^{k+1}) \right\} \right) \right] + (1 - \alpha(k)) W_i^k(p|s_i^k). \quad (2)$$

where β is the discount rate. Otherwise $W_i^{k+1}(p|s_i^k) = W_i^k(p|s_i^k)$.

Equation (2) implies that the change in perceived value of play ($W^{k+1}(\cdot) - W^k(\cdot)$) is α_k times the difference between the observed value of the k th period's outcome from choosing p , computed as profits plus the discounted expected continuation value, and the initial (or k th period) perceived value of the action. So $\alpha(k) \in [0, 1)$ determines the impact of current observations on perceived values.

When *asynchronous* updating is used, $W_i^k(p|s_i)$ is updated only for $p = p_i^k$ at $s_i = s_i^k$. In this case, $\pi_i(s_i^k, p)$ is the realized profits of firm i in iteration k , and this is the only object the algorithm needs to know to do the update. But when *synchronous* updating is used, $W_i^k(p|s_i)$ can be updated for all $p \in P$ at $s_i = s_i^k$. In our extreme case, we assume the algorithm can calculate the $\pi_i(p, p_j^k)$ exactly. We call this *perfect synchronous* updating. This would be possible if p_j^k is observed by the algorithm, and it knows the market demand function and its own costs, which is the case in the initial perfect synchronous algorithm we compute. This coding of the *perfect synchronous* and the *asynchronous*, that is endowing the perfect synchronous algorithm with the ability to compute the profits that would have been earned had it taken a different action but assuming the asynchronous algorithm can only learn from the profits generated by the action taken, makes it easy for us to illustrate the issues we focus on.

For both the perfect synchronous and asynchronous algorithms, we need an initial set of perceptions, a $\mathcal{W}_i^0 = \{W_i^0(p|s_i)\}_{p \in P, s_i \in \mathcal{S}}$. For all the cases we consider in this paper, we take this to be a random draw from the same distribution (specified below).

2.2.1 | Baseline parameterization

In our base case, $\mathcal{S}_i$ is a singleton, $\alpha(k) = \alpha$, $\beta = 0$, and both firms have the same profit function, resulting in the updating rule (Equation 2) reducing to

$$W_i^{k+1}(p) = \alpha \pi(p, p_j^k) + (1 - \alpha) W_i^k(p). \quad (3)$$

After illustrating what happens in this case, we move to more general cases.

More generally the possible updating processes would depend on (i) what the algorithm observes about competitors' play, and (ii) what it knows about the primitives of the game (in our case the demand and cost functions). A reinforcement learning algorithm with perfect synchronous updating can be quite sophisticated. In every period, it updates the values of all feasible prices at the current state and so necessarily considers the counterfactual profits it would have received had it chosen an alternate price. To correctly calculate these counterfactuals it would have to have a model of demand and costs, which is the case in the perfect synchronous updating model that we focus on. By contrast, an algorithm with asynchronous updating is rather simple; all the updating process does is use its current price and profits to compute an update on the values of the profits it expects to receive from that price.

Between the perfect synchronous and asynchronous updating there is room for protocols that incorporate more or less information and approximations that are understood to be noisy or biased. In the example we illustrate below, the firm knows its own costs and quantity, does not know the price of its competitors or the form of the demand function, but does know that its residual demand is downward sloping in its own price. It updates the $W_i^k(p|\cdot)$ at all prices less than the price played whose value can be rationalized only by quantities less than the quantity it received, and it updates all $W_i^k(p|\cdot)$ at higher prices which require quantities greater than the quantity it received.

A comparison between possible updating rules is really a comparison of the sophistication of the AI algorithms the firms employ. This is likely to depend on the complexity of their economic environment and the willingness to devote resources to supporting the AI program underlying the algorithm.

3 | REST POINTS OF DIFFERENT AI ALGORITHMS

We begin with computational results from the Bertrand pricing game introduced in Section 2.1. Firms have a discount rate of zero, so they do not care about future payoffs, and the state space is a singleton, so history-dependent strategies are not feasible. Note that either a discount rate of zero or an inability to formulate history-dependent strategies makes repeated game interactions that could support prices above a static Nash-in-price equilibrium, like trigger price strategies, unattainable.

3.1 | Computational results

This section provides computational results for two homogenous product static Bertrand games; one game uses an AI algorithm that updates synchronously and the other asynchronously. Both algorithms use the updating rule in Equation (3) with $\alpha = 0.1$ when updating, and take “greedy” actions (i.e., they always chose the price which their perceptions indicate has the highest profits). Both algorithms also use the same procedure to obtain their initial perceptions of the value associated with all feasible actions.

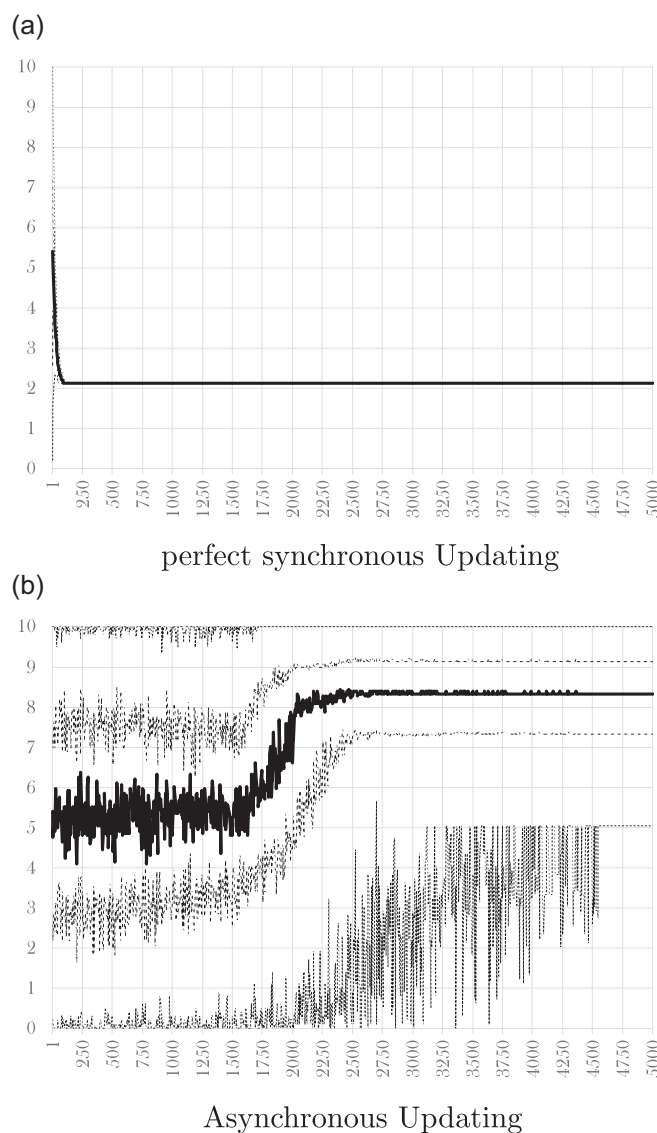
Our parameterization has demand equal to zero for any price above 10 (which is also the monopoly price), and equal to one for any price below or equal to 10. Marginal cost is equal to two. Feasible prices are given by a grid with 100 elements, equally spaced between 0.01 and 10, inclusive. There are two Nash equilibria to this game—one in which both firms play 2.03 and the other in which both firms play 2.13. For initial perceptions of the values associated with each price, we draw from a uniform distribution with endpoints 10 and 20 (or $U[10, 20]$). The figures we present contain quantiles of the distribution of a 100 price paths we generate in this way.

Note that the initial perception of profits from playing any given price is always above any possible profit outcome (though the extent to which they are above is random). As is well known in the reinforcement learning literature, the fact that each sample path starts with initial values that are higher than any possible outcome induces exploration.²⁷ We deal with different initial conditions distributions when we consider formal properties of the two updating algorithms below.

Figure 2 summarizes pricing outcomes for the baseline parametrization described in Section 2.²⁸ Figure 2a provides quantiles of the price paths from an algorithm when both firms employ perfect synchronous updating; Figure 2b shows price paths when both employ asynchronous updating. There are five lines in each panel of Figure 2. From bottom to top these are, by period, the min, 25th percentile, median, 75th percentile, and max price across the 100 simulations that are run.

Start with Figure 2b. Convergence is relatively slow, but after 4600 periods none of the paths simulated from the AI algorithm with asynchronous updating change their preferred actions and all of the percentiles stay constant.²⁹ Most notably there is a distribution of rest point prices, but *all* are significantly higher than the Nash equilibrium prices. The

FIGURE 2 Price outcomes with different algorithm designs (a) perfect synchronous updating, (b) Asynchronous updating. Prices (vertical axis) by period (horizontal axis) from 100 simulations are shown. The lines, from bottom to top, represent the min, 25th percentile, median, 75th percentile, and a max of the distribution of prices in each period. Results are for a static Bertrand market with two firms selling homogeneous goods. Results are shown for firm 1. The model is parametrized as follows. Demand, $Q = 1$ if $p \leq 10$, zero otherwise. Marginal cost = 2. Feasible prices exist on a grid with 100 elements equally spaced between 0.1 and 10 inclusive. Firms put a zero weight on future profits (the future is discounted to zero). The state space is a singleton. The weight on current returns in updating is given by $\alpha = 0.1$. Initial conditions are i.i.d. draws from $U[10, 20]$, for each $W(p)$ for each firm. For the core code see Appendix A.1.



median price is 8.34 and the minimum is 5.06. Note also that all quantiles tend to increase over the course of the learning process; that is, they move away from the Nash equilibria.

By contrast, when the AI learns via perfect synchronous updating prices converge quickly to Nash pricing levels. In this instance all prices, across all 100 simulations settled on 2.13 after approximately 105 iterations. Thus learning was much faster, price outcomes corresponded to a static Nash outcome, and initial conditions had no influence on the limiting prices (there is no variance in the final prices over the 100 simulations).

Figure 2 illustrates the importance of the learning (or updating) process in understanding the implications of price competition between AI algorithms on the outcomes that may emerge. Asynchronous updating leads to supra-competitive ("high") prices. perfect synchronous updating leads to pricing that is in line with what economists would think of as a competitive outcome (the static Nash equilibrium in prices). Importantly, this happens in a setting in which firms do not care about the future and are unable to play history-dependent strategies. Thus though standard collusive equilibria of the sort familiar from the repeated game literature are not feasible, the asynchronous algorithm always generates rest point prices that are significantly above the Nash equilibrium values.

Figure 3 provides more detail on the learning process. Each panel in Figure 3 shows the convergence process for a set of initial conditions drawn from $U[10, 20]$. As in Figure 2a, corresponds to the perfect synchronous updating case and Figure 2b corresponds to the asynchronous updating case. Each panel shows the prices chosen by firm 1's AI with circles. The hollow circles indicate chosen prices for which the $W^k(p)$ is not updated upward; solid circles indicate chosen prices for which the $W^k(p)$ is updated upward.

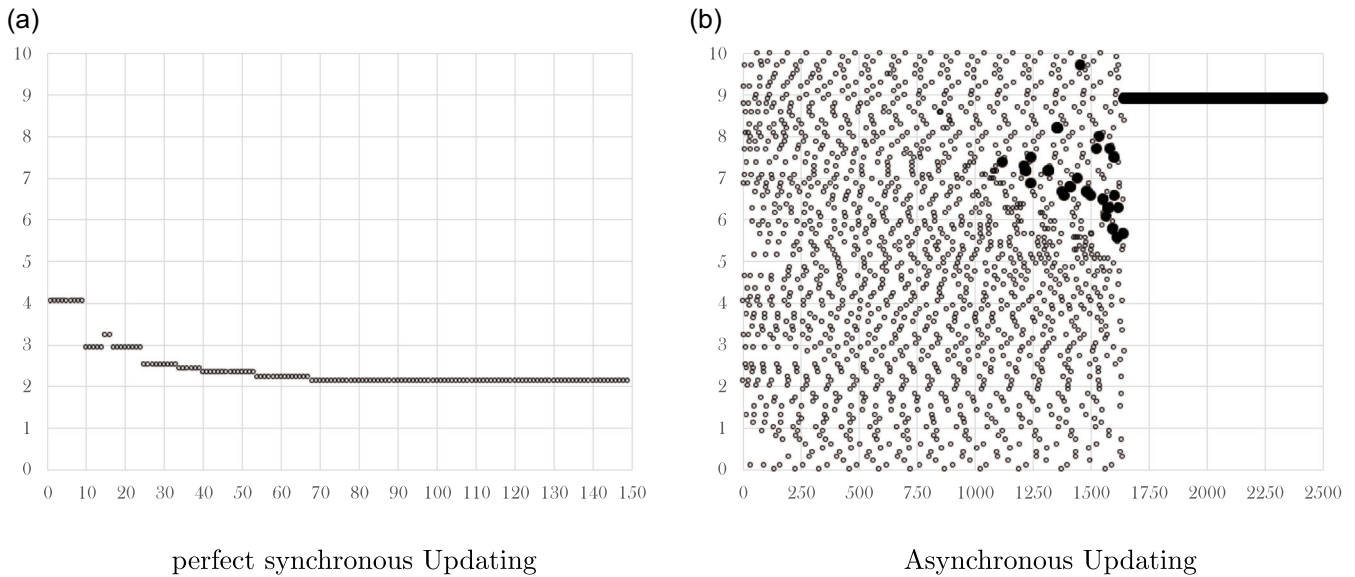


FIGURE 3 Convergence with different algorithm designs (a) perfect synchronous updating, (b) Asynchronous updating. Prices (vertical axis) by period (horizontal axis) are shown for a firm 1, in a static Bertrand market with two firms selling homogeneous goods. Hollow circles indicate (chosen) prices for which the $W(p)$ is updated downward. Solid circles indicate (chosen) prices for which the $W(p)$ is updated upward. The model is parametrized as per Figure 2.

Figure 3a, the perfect synchronous case, convergence is fast. The Nash pricing outcome of 2.13 is reached by period 70. Initially, all perceptions are higher than the profits that result from the price chosen. This leads to the updating process (3) in the perfect synchronous case adjusting perceptions of the profits from *all* prices downward. For example, in iteration 1, initial conditions on $W(p)$ lead firm 1 to choose $p = 4.05$, and its rival chooses 0.92. Firm 1 does not earn any profits from this price combination. In iteration 3, after $W(p)$'s have been updated twice, the rival chooses 3.14. Next, in round 10, firm 1 changes its choice to 2.94, just undercutting 3.14. This, however, coincides with the rival raising its chosen price to 3.84 (which more profitably undercuts firm 1's original choice of 4.05). This combination of chosen prices continues until iteration 15, when firm 1 raises its chosen price to 3.24. After one iteration of updates to the rival's $W(p)$, 3.14 becomes the rival's best choice. Then, in iteration 17, the cumulative updates of firm one's $W(p)$'s are such that 2.94 becomes the best price. From that point on, a pattern of intermittent undercutting develops until the prices converge. This convergence pattern is common, with the influence of initial conditions in early periods leading to a small amount of nonmonotonicity, as seen here.

Figure 3b (the asynchronous updating case) convergence is nonmonotonic and slow. The prices converge to 8.89 in period 1645. As in the perfect synchronous case, the fact that the initial condition draws are higher than monopoly profits, implies that in early periods, the evaluations fall. That is, when a price p^k is chosen, it generates a lower profit than the $W^k(p)$ associated with p^k , and so $W^k(p^k)$, and in this asynchronous algorithm only $W^k(p^k)$, is adjusted downward. Eventually, the initial valuation of some other price is higher than the profit earned from the given price, so some other price will be chosen. Since only the price that is chosen has its $W^k(p)$ updated, and the second price also starts from a perception which is higher than monopoly profits, it takes quite a few iterations before the chosen price can generate a profit which is higher than the iteration's perception of its value.

The first time the chosen price generates a profit that is higher than its initial valuation, that is, $W^k(p^k)$, is in iteration $k = 1, 119$. This leads to an upward adjustment and a solid black circle in the bottom panel of Figure 3. As long as the price of the rival is not reduced to match or undercut the price, firm i does not change its price. Since at $k = 1119$ the rival's price is greater than $p^{1,119}$, it obtains no profits and its valuation of its chosen price falls. Eventually it chooses a different price which, if equal to or lower than $p^{1,119}$, generates positive profits. If the rival's new price is lower, firm i 's profits go to zero and its evaluation of $p^{1,119}$ falls. However, if the rival happens to chose $p^{1,119}$ both firms earn positive profits. If, in addition, at that k , $\pi(p^k, p^k) \geq \max[W_i^k(p), W_j^k(p)]$, the algorithm will have converged to p^k .

In this example, the two firms matched on price 22 times before the second requirement for convergence. The asynchronous updating and Equation (3) insures that for $k > 1, 119$ the absolute value of both $[W_i^k(p^*) - \pi(p^*, p^*)]$ and $[W_j^k(p^*) - \pi(p^*, p^*)]$ converge monotonically to zero, and if $p \neq p^*$, $W^k(p)$ remains forever at $W^{k=1,119}(p)$.

3.2 | Theoretical results

It should be clear that the price the asynchronous algorithm rests at depends on initial conditions, hence the distribution of converged values illustrated by the quantiles of the sample paths in 2. Indeed, provided the distribution of initial conditions is sufficiently rich, the limiting outcome from the asynchronous algorithm has a positive probability of any $p \in \mathcal{P}$. By contrast, the perfect synchronous algorithm can only converge to a Nash equilibrium. The following provides a formal justification of these points, and establishes that the computational results reflect intrinsic differences in how asynchronous and perfect synchronous updating function.

Proposition 1. *Consider the duopoly pricing game between AI algorithms described above with initial conditions drawn randomly from a distribution such that for any p , say p^* , there is a positive probability that $W_i^0(p^*) > W_i^0(p) \forall p \neq p^*$, and $W_i^0(p^*) < \min_{p_j} \pi(p^*, p_j)$.*

1. *If the AI algorithms use perfect synchronous updating*
 - (a) *and the two algorithm choose a pair of prices which is a Nash equilibrium to the static pricing game, they will continue to charge these prices in every future period, and, in addition,*
 - (b) *any rest point of the algorithm must be a Nash pricing equilibrium of the duopoly game.*
2. *If the AI algorithm uses asynchronous updating, there is positive probability that the algorithm converges to any $p \in \mathcal{P}$.*

Proof. (1a). Assume that at period k , which is the initial condition for our inductive argument, the two algorithms charge a pair of prices (p_1^N, p_2^N) which is a Nash equilibrium of the duopoly pricing game. This implies that $\pi_1(p_1^N, p_2^N) \geq \pi_1(p, p_2^N)$ for every $p \in \mathcal{P}$. The updating of the values of the algorithm of firm 1 at period k is done for every $p \in P$ by the updating rule $W_1^{k+1}(p) = \alpha \pi_1(p, p_2^N) + (1 - \alpha) W_1^k(p)$ (and analogously for firm 2). Given that $\pi_1(p_1^N, p_2^N) \geq \pi_1(p, p_2^N)$ and that $W_1^k(p_1^N) = \max_{p \in P} \{W_1^k(p)\}$, $W_1^{k+1}(p_1^N) = \max_{p \in P} \{W_1^{k+1}(p)\}$ and therefore the algorithm will choose p_1^N at iteration $k + 1$. The same holds for the second firm, and therefore the prices at iteration $k + 1$ are (p_1^N, p_2^N) , which proves the inductive step of the argument. $\square$

Proof. (1b). Assume to the contrary that there is a k^* such that $\forall k > k^*, W^k(p'_1) = \max_p W_1^k(p)$ with the analogous condition holds for firm j , and let $b_1(p'_2)$ be the best response of player 1 in the duopoly pricing game.³⁰ For the contrary assertion to be true $b(p'_2) \neq p'_1$ and $\pi_1(b(p'_2), p'_2) > \pi_1(p, p'_2)$ for every $p \in P$. With synchronized updating $W_1^{k+1}(p) = \alpha \pi_1(p, p'_2) + (1 - \alpha) W_1^k(p)$ for every $p \in P$. Consequently, $\lim_{k \rightarrow \infty} W_1^k(p'_1) = \pi(p'_1, p'_2) < \lim_{k \rightarrow \infty} W_1^k(b(p'_2)) = \pi(b(p'_2), p'_2)$, a contradiction. $\square$

Proof. (2). Sufficient conditions for p^* to be a rest point to the game when updates are asynchronous are: (i) $W_i(p^*) \geq W_i(p)$ for every $p \neq p^*$ and $i = 1, 2$, and (ii) $\pi_i(p^*, p^*) \geq W_i(p)$ for every $p \neq p^*$ for $i = 1, 2$. Under condition (i) the firms indeed choose the price p^* and condition (ii) guarantees that condition (i) will continue to hold in all future periods. For any p^* there is a positive probability that the random draw on the initial conditions satisfies both (i) and (ii). $\square$

Remark. None of the three parts of this proposition depend on the exact structure of the profit function, the updating function provided $\alpha \in (0, 1)$, or the initial condition distribution provided that it satisfies our “sufficiently rich” support condition. As a result, though we have computed results for the same two algorithms from the logit differentiated product case, for quantity-setting games and for different initial conditions distributions we omit most of them from the main text. We discuss quantity-setting (Cournot) in Appendix A.4 and logit results in Appendix A.2.

4 | EQUILIBRIA FOR ALGORITHMIC PRICING GAMES

The rest points of the asynchronous and perfect synchronous algorithms satisfy different equilibrium conditions, and an understanding of those conditions clarifies why and when different learning rules imply different rest points for AI pricing games. Unless otherwise stated, the results in this section apply to any pricing algorithm that abides by the general description of the algorithms given in Section 2.2. The relevant equilibrium conditions are provided in Fershtman and Pakes (2012), and Asker et al. (2020) and the reader interested in a more formal analysis of their implications should consult those papers.

In the general case, the algorithm's choice of actions, the choice of $p \in \mathcal{P}$, can differ with $s \in S$ (for e.g., as a function of past prices). If the algorithm has converged, those policies are constant thereafter for each $s \in S$. Any set of such policies generates a Markov chain on S ; that is, the state at iteration k together with the prices chosen at that iteration generate a transition to the state at iteration $k + 1$. Any finite state Markov chain will eventually wander into a recurrent subset of the points in S , say $\mathcal{R} \subset S$, and stay within it forever. So all rest points of AI pricing games are contained in the recurrent class of the Markov process generated by the policies that the AI converged to. Since S_i can include functions of past prices, this includes policies with history-dependent strategies.

The equilibrium concepts we consider are all refinements of experience-based equilibrium (EBE). An EBE has three components:

- a subset of the state space, denoted by $\mathcal{R} \subset S$.
- strategies for each i , say $\{p_i^*(s)\}$ for every $s \in S$, and
- a set of values for each i , our $\{W_i(p|s)\}_{p,s}$, that have the interpretation of beliefs about the expected discounted value of profits were the agent to play price p_i at state s_i .

For these objects to satisfy the conditions of an EBE it must be the case that

1. the strategies are optimal given the beliefs embodied in the $\{W_i(p|s)\}$, that is, $p_i^*(s) = \arg \max_{p \in \mathcal{P}} \{W_i(p|s)\}$ for all i and all $s \in S$,
2. $\mathcal{R}$ is a recurrent class of the Markov process generated by these strategies, and
3. at all $s \in \mathcal{R}$, $W_i(p_i^*(s)|s)$ does in fact equal the expected discounted value of profits if the policies are followed.

The rest point of an AI pricing game that plays greedy policies and uses asynchronous updating will satisfy the conditions of an EBE. The fact that the policies are greedy insures (1), and if we are at a rest point that point is a recurrent class (insuring 2).³¹ The third condition states that if equilibrium policies are played indefinitely the average of the realized discounted value at each $s \in \mathcal{R}$ converges to $W_i(p_i^*(s)|s)$. Given the updating equation (Equation 2), the fact that we stay at the rest point indefinitely (so $s = \mathcal{R}$), and the fact that $(p_i^*(s), p_j^*(s))$ is played repeatedly, insures that $W_i(p_i^*(s)|s)$ converges to $[1 - \beta]^{-1} \pi_i(p_i^*(s), p_j^*(s))$, the discounted value of returns that would be earned if $(p_i^*(s), p_j^*(s))$ were played indefinitely. An analogous condition holds for firm j .

Notice that EBE does not put as stringent a condition on the perception of returns for feasible policies not played: for policies “off the equilibrium path.” It requires only that the perceptions of these returns are less than those of the optimal policy; that is, that $W_i(p|s) \leq W_i(p_i^*(s)|s)$. The implication is that if randomness in the initial conditions happens to generate high perceived values for both agents at a particular couple of choices, and those values generate profits plus discounted continuation values which are higher than the perceived values at other prices, the values at the other points will never be updated. Then the algorithm will stick with the values that generated the initial choice forever. If the initial iteration's values did not satisfy these conditions, a subsequent iteration could, and we would reach high prices at a later iteration, as in Figure 1b.

Perfect synchronous updating leads to rest points that satisfy stronger conditions than those of an EBE. In particular, the fact that all values are updated at each iteration also restricts the perceptions of returns for the feasible policies that are not played at the rest point. The implications of the additional conditions are different when the model has $\beta = 0$ and when $\beta > 0$.

When $\beta = 0$, the update for perceptions at the rest point for firm i is just current returns. With perfect synchronous learning, this implies that the update for every $p \in \mathcal{P}$ for firm i is $\pi_i(p, p_j^*)$ and for agent j it is $\pi_j(p, p_i^*)$. Since the rest point is visited repeatedly, the perceived value of play at every p converges to the discounted value of these profits for

the two firms. As a result, regardless of initial conditions, the nature of S , or the form of the profit function, the optimal strategy at the rest point must eventually satisfy $\pi(p_i^*, p_j^*) = \max_{p \in P} \pi_i(p, p_j^*)$ with an analogous condition for firm j . So when $\beta = 0$ the rest point must be a Nash equilibrium, as in Figure 1a.

When $\beta \neq 0$, the rest point from a perfect synchronous updating algorithm need not satisfy the condition that the perception of the value of all feasible policies is equal to the expected discounted value obtained from playing the action repeatedly. When $\beta \neq 0$, the rest point satisfies the second set of equilibrium conditions introduced in Fershtman and Pakes (2012); those of a restricted EBE (or REBE). REBE abides by the first two conditions of an EBE, but strengthens the consistency requirement (the third condition) by requiring that the consistency condition hold for feasible actions that are not optimal (for $p(s_i) \neq p^*(s_i)$), but only when the feasible actions result in outcomes that are in $\mathcal{R}$ (i.e., are visited repeatedly).

The restriction of the consistency condition to outcomes that are in $\mathcal{R}$ is also the rest point condition that emanates from perfect synchronous AI pricing algorithms.³² This is because the discounted value of future returns for points outside of $\mathcal{R}$ are not visited repeatedly, so we cannot use a law of large numbers to prove that their perceived values converge to any particular value. When $\beta \neq 0$ and a perfect synchronous learning model is used, the update for a feasible but nongreedy policy consists of current profits plus the discounted perceived value of future play were the nonoptimal action played. However since the nongreedy policy is not played at the rest point, though the perfect synchronous algorithm updates current profits with realized profits at the rest point, the discounted perceived values of future play (the “continuation values”), are not updated with realizations from what would have occurred were those actions taken. So if greedy policies are played throughout, the perceived values for feasible actions at the rest point will be determined by the random draw on initial conditions, while if there is some experimentation, the draws on the experiments will also help determine the rest point.

Asker et al. (2020) consider a condition which restricts the perceptions of returns from off the equilibrium path play further and use it in their analysis of dynamic procurement auctions. However their extra condition, labeled “boundary consistency,” is not used in any AI pricing game we are aware of, and will not be satisfied “naturally” at a rest point of the algorithm (to check it a separate subroutine must be built into the algorithm). Consequently, we focus on the implications of the simpler REBE condition in the extensions investigated in the next section.

5 | EXTENSIONS TO MORE COMPLEX ENVIRONMENTS

This section considers the implications of learning models when our basic AI pricing algorithm is extended in various ways. Given the results above, we do this first for the case where $\beta = 0$ and then for the case where $\beta > 0$. The extensions include allowing for more than two competitors, experimentation, and cases where the algorithm only has the ability to update the value of counterfactual policies imperfectly.

The latter case is relevant when either competitor's prices are not observed or the demand system is not known. Then the algorithm might still use basic economic reasoning, in our example the assumption that the residual demand curve slopes downward, to update the value of counterfactual policies. This example is extreme in that it assumes that neither the competitor's prices nor the demand system can be even imperfectly inferred from observed behavior. However, it will suffice to make the point that adding a little bit of information to the algorithm, in addition to the profits the algorithm “observes,” makes notable changes to the rest point.

When considering models with $\beta > 0$, we allow for history-dependent strategies. This enables the algorithm to generate equilibria that mimic collusive equilibria supported by familiar punishment strategies. Two points should be kept in mind here. First, the strategies generated by the algorithm are partly a function of random draws, in contrast to purposeful behavior. Second, models with larger state spaces are harder to compute. The complexity of an algorithm which keeps past prices (or any other competitor-specific state) in memory increases, often dramatically, with the number of competitors, and this limits their usefulness.³³

5.1 | $\beta = 0$: Algorithms that play static games

5.1.1 | Increasing N , the number of firms

Table 1 extends the computational results reported in Figure 2 by varying N , the number of firms. Both the perfect synchronous and asynchronous algorithms are proficient at finding the optimal monopoly price when $N = 1$. As N increases to 2 and higher, the perfect synchronous algorithm converges quickly to Nash pricing outcomes.³⁴

TABLE 1 Pricing outcomes as N varies.

Algorithm	N	Min	25th	Median	75th	Max	% >static Nash	Time to converge
Perfect synchronous	1	10.00	10.00	10.00	10.00	10.00	–	50
	2	2.03	2.13	2.13	2.13	2.13	0	115
	3	2.03	2.13	2.13	2.13	2.13	0	70
	4	2.03	2.13	2.13	2.13	2.13	0	75
	5	2.03	2.03	2.03	2.03	2.03	0	85
	10	2.03	2.03	2.03	2.03	2.03	0	90
Asynchronous	1	10.00	10.00	10.00	10.00	10.00	–	1235
	2	4.65	7.28	8.39	9.29	10.00	100	4550
	3	2.13	2.13	2.84	3.95	10.00	71.5	8530
	4	2.03	2.13	2.13	2.13	7.38	19.9	10,395
	5	2.03	2.03	2.03	2.03	3.34	12.1	10,395
	10	2.03	2.03	2.03	2.03	2.23	0.1	10,025

Note: For each N , 1000 simulation runs were conducted. The minimum, 25th percentile, median, 75th percentile, and maximum of the prices for firm 1, across the 1000 simulations, once all simulations have reached a rest point, is reported. “% > Static Nash” reports the percent of simulations that reach a rest point that is higher than the highest static Nash price. For $N \in \{2, 3, 4\}$ the highest static Nash price is 2.13 (2.03 is also a Nash outcome). For $N \geq 5$ the only static Nash equilibrium price is 2.03. “Time to converge” reports the number of iterations until the reported price moments cease to change (each simulation was eventually stopped after 40,000 iterations). The model is as per Figure 2, but for the reported changes in N .

By contrast, the asynchronous algorithm converges to a price greater than Nash in 100% of instances when $N = 2$, and 71.5% of instances when $N = 3$. This decreases as N increases further, with this percentage approaching zero by the time $N = 10$. Further, the number of iterations until convergence for the asynchronous algorithm is typically two orders of magnitude higher than for the perfect synchronous algorithm.³⁵

The results in Figure 2 indicate that having multiple competitors can significantly mitigate any adverse price effects that arise from the use of an asynchronous algorithm. That said, even in only moderately concentrated markets (those with 4–6 competitors), pricing above static Nash levels can occur relatively frequently.³⁶

5.1.2 | Experimentation

Thus far we have investigated algorithms that do not experiment as they learn. If an algorithm is programed to experiment, it deviates from choosing the greedy policy, that is, the policy associated with the maximum of the W ’s in memory, and instead chooses a random action and obtains information on what the payoff of that action is. We consider how experimentation might affect the asynchronous results in Figure 2.³⁷

Deciding how an asynchronous algorithm may experiment involves making a range of decisions. We consider experimentation which occurs with probability $[k^\frac{1}{\theta}]^{-1}$ in each round, where k indexes the iteration of the simulation, and θ is a parameter we vary. So the frequency of experimentation declines as k increases and the algorithm has more observations to learn from. For a given k , the frequency of experimentation increases with the θ parameter. When the algorithm does experiment, it chooses each feasible action with equal probability.

There will not be a rest point after a finite number of iterations without an end to experimentation. Since the asynchronous algorithm, absent experimentation, reaches a rest point within 5000 iterations (see Figure 2), we allow experimentation to occur for 10,000 iterations. That is, we allow experimentation to double the computational time required by the algorithm to reach a rest point when there is no experimentation. After 10,000 iterations, we stop experimentation and let the algorithm run until it converges.

Table 2 reports the resulting price distributions as θ , the intensity of experimentation, varies. For each value of θ we also report the minimum of the number of times any action is attempted during the course of learning, labeled “Min # of times an action is played.” This provides a way to judge the extent to which experimentation reaches all feasible actions.

TABLE 2 Pricing rest points as θ varies.

θ	Min # of times an action is played	Pr(experimentation) at iteration 10,000	Min	25th	Median	75th	Max
1	14	0.0001	5.36	7.07	8.03	9.24	10.00
2	17	0.0100	3.04	6.42	7.68	8.89	10.00
3	20	0.0464	3.04	5.46	7.02	8.54	10.00
4	24	0.1000	2.73	4.25	6.27	7.98	10.00
5	29	0.1585	2.53	3.84	5.56	7.68	10.00
6	33	0.2154	2.53	3.54	5.01	7.28	10.00
7	37	0.2683	2.13	3.44	4.65	6.87	10.00
8	42	0.3162	2.13	3.34	4.40	6.57	10.00
9	45	0.3594	2.13	3.34	4.20	6.37	10.00
10	48	0.3981	2.13	3.34	4.15	6.06	10.00

Note: For each θ , 100 simulation runs were conducted. Experimentation occurs with probability $k^{-\frac{1}{\theta}}$ in each round, where k indexes the iteration of the simulation. If experimentation occurs, an action is selected at random (with each possible action having equal probability). Experimentation stops after 10,000 iterations. The algorithm continues after that point with no experimentation until a rest point is reached. The minimum, 25th percentile, median, 75th percentile, and a maximum of the prices for firm 1, across the 100 simulations, once all simulations have reached a rest point, is reported. $N = 2$. The model is as per Figure 2, but for the addition of experimentation.

The results in Table 2 indicate that experimentation is not a simple fix for the propensity for the asynchronous algorithm to elevate prices. Apparently a reasonable amount of experimentation does mitigate this propensity, but does not remove it. What an optimal experimental process might look like is unclear as it would have to face the familiar problem of trading off “exploration” against “exploitation” in an environment with competition.³⁸ It must surely penalize excessive computational burden, and occur at some frequency which is both frequent enough to be informative, but not so frequent as to cause an expected discounted profits decline. What does seem clear is that algorithms that build in experimentation are not likely to completely fix the propensity of asynchronous algorithms to elevate prices.

5.1.3 | Imperfect counterfactual updating

As previously noted, the asynchronous and perfect synchronous algorithms require access to very different amounts of information; the asynchronous algorithm needs to know only the profits it received from the price it played, whereas the perfect synchronous algorithm requires an ability to run a complete set of counterfactuals conditional on the state. To obtain accurate counterfactuals the algorithm needs to observe competitors' prices and then be able to map them, together with different selections of its own price, into profits (in our example this would require knowledge of the residual demand curve). We have endowed our perfect synchronous algorithm with this ability, but in many actual markets accurate counterfactual updating may be impossible.

We now explore the behavior of algorithms that have access to less information than our perfect synchronous but more information than our asynchronous algorithms. In particular, we leverage a relatively uncontroversial additional bit of information: the knowledge that residual demand curves slope (weakly) downward. That information allows us to employ a less accurate form of synchronous updating. Hence, we call this *synchronous updating using downward demand*.

Recall that $d_i(p^*, p_j^k)$ is our notation for the demand received by the firm at the price it chose in iteration k . Our counterfactual updating rules are modified as follows. If $p > p^*$ and the current iteration's perception of the value at p is higher than what could be rationalized by assuming demand was given by current demand, then we update the value of playing the counterfactual price assuming that $d_i(p, p_j^k) = d_i(p^*, p_j^k)$, so that $W_i^k(p)$ is adjusted downward. Similarly, if $p < p^*$ and $W_i^k(p)$ is lower than what could be justified by assuming that $d_i(p, p_j^k) = d_i(p^*, p_j^k)$, then $W_i^k(p)$ is adjusted upward by assuming that $d_i(p, p_j^k) = d_i(p^*, p_j^k)$. Formally the updating rules for different p are³⁹

$$\begin{aligned}
p &= p^*, W_i^{k+1}(p^*) = \alpha(p - c)d_i(p^*, p_j^k) + (1 - \alpha)W_i^k(p), \\
p &> p^*, W_i^{k+1}(p) = \alpha \min\{(p - c)d_i(p^*, p_j^k), W_i^k(p)\} + (1 - \alpha)W_i^k(p), \\
p &< p^*, W_i^{k+1}(p) = \alpha \max\{(p - c)d_i(p^*, p_j^k), W_i^k(p)\} + (1 - \alpha)W_i^k(p).
\end{aligned} \tag{4}$$

Figure 4 shows the impact of taking the model in Figure 2b (static Bertrand with asynchronous updating) and allowing updating to occur in instances when the underlying values violate the assumption of downward-sloping demand. The minimum rest point is 2.03 and the maximum is 3.84. The reason that the outcomes are markedly different to those in Figure 2 is that, by imposing the additional demand structure, W 's attached to high prices get updated downward frequently in early iterations (i.e., given that W 's are initialized at high values, in early iterations this happens whenever a lower price is chosen). This means that the actions actually chosen, by both firms, through the course of the learning process are much more likely to be relatively low. The outcomes reported in Figure 4 suggests that leveraging minimal assumptions about the economic environment can have a significant mitigating impact on any tendency for an asynchronous algorithm to generate supra-competitive prices.⁴⁰ Leveraging the downward-sloping demand assumption also dramatically increases the speed with which the algorithm reaches a rest point. The speed increase is of at least an order of magnitude.

The example shown here, in which the assumption of downward demand is explored, indicates that having a complete model of the economic environment is not necessary to significantly mitigate the propensity for an asynchronous algorithm to generate high prices. Depending on the setting, other justifiable assumptions may be able to be leveraged, whether in addition to downward demand or as a substitute for it.⁴¹

5.2 | $\beta > 0$: Algorithms that consider future returns

When $\beta = 0$, pricing algorithms that use asynchronous learning are expected to generate rest points with supra-competitive pricing, but those that used perfect synchronous learning will not. These results are independent of the details of the algorithm, including the structure of the profit function and the states that strategies can condition on. However, the result for perfect synchronous learning does not generalize to models with $\beta > 0$.

Recall that when an algorithm converges to a rest point, that point is the only point in the recurrent class, and the conditions that the policies and perceived values a perfect synchronous learning algorithm must satisfy at the rest point are those of a REBE. If $\beta > 0$, the feasible but nonoptimal policies at the rest point are updated using a correct measure of current profit but a discounted continuation value which is not updated because the policies that are not optimal are never actually played. To explore the implications of this fact in a way that makes their relationship to the literature on repeated games transparent, we consider models with history-dependent strategies.

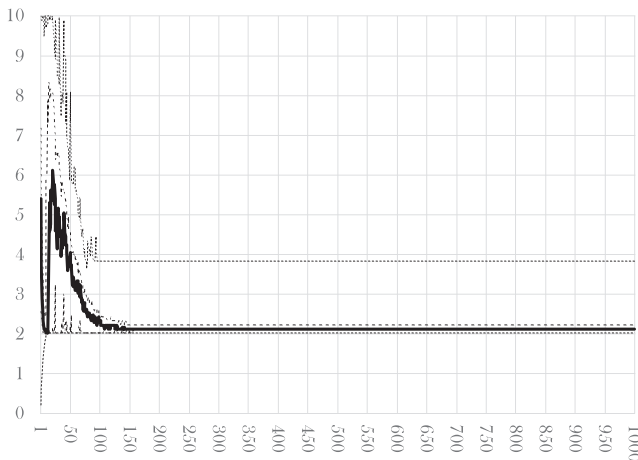


FIGURE 4 Price outcomes with synchronous updating using downward demand. Prices (vertical axis) by iteration (horizontal axis) from 100 simulations are shown. The lines, from bottom to top, represent the min, 25th percentile, median, 75th percentile, and max of the distribution of prices in each period. The model mirrors that in Figure 2a, but for the addition of the ability of the algorithm to update all $W(p)$'s using the assumption that demand slopes (weakly) downward.

To see the possible implications of not updating continuation values, it suffices to consider a model where there are two possible actions and a state space that consists of last period's prices, that is

$$\mathcal{P} = (p^1, p^2), \text{ and } \mathcal{S} = \{(p^1, p^1), (p^1, p^2), (p^2, p^1), (p^2, p^2)\}.$$

We consider the conditions that need to be satisfied for (p_i^1, p_j^1) to be a rest point of this algorithm.

Since policies have converged, Equation (2) guarantees that perceived values will also. Since states other than (p_i^1, p_j^1) are not recurrent, the value associated with play at those points need not equal the discounted value that would result were those states being visited repeatedly. The REBE conditions for the rest point imply only that in the limit

$$\begin{aligned} W_i(p^1|p_i^1, p_j^1) &= \pi_i(p_i^1, p_j^1) + \beta W_i(p^1|p_i^1, p_j^1) \Rightarrow W_i(p^1|p_i^1, p_j^1) = \pi_i(p_i^1, p_j^1)[1 - \beta]^{-1}, \\ W_i(p^2|p_i^1, p_j^1) &= \pi_i(p_i^2, p_j^1) + \beta \max_{y \in \mathcal{P}} W_i(y|p_i^2, p_j^1) < W_i(p^1|p_i^1, p_j^1), \end{aligned}$$

and the analogous conditions for firm j .

So for (p_1, p_1) to be a rest point, all we require is

$$\left[\pi_i(p_i^1, p_j^1) - \pi_i(p_i^2, p_j^1) \right] > \beta \left(\max_{y \in \mathcal{P}} W_i(y|p_i^2, p_j^1) - [1 - \beta]^{-1} \pi_i(p_i^1, p_j^1) \right),$$

and the analogous condition for firm j . ($W_i(p_1|p_i^2, p_j^1)$, $W_i(p_2|p_i^2, p_j^1)$) are not constrained to be consistent with the returns that would actually be earned were those policies actually played. Rather they are determined by the random draws—either just those on initial conditions or, if there is experimentation, in conjunction with the draws from experimenting.

These equilibrium conditions also determine what would happen were we to force the algorithm to engage in “off the equilibrium” path behavior, for example, if we used the policies to simulate a path in which firm i chose price p_2 . If $\max_{y \in \mathcal{P}} W_j(y|p_i^2, p_j^1) = W_j(p_2|p_i^2, p_j^1)$ firm j would respond in the next period with p_2 . If this occurred and $\max_{y \in \mathcal{P}} W_i(y|p_i^2, p_j^2) = W_i(p_2|p_i^2, p_j^2)$, then firm i would chose p^2 in the next period, and if the analogous condition held for firm j both firms would choose p^2 in all subsequent periods. That is (p_1, p_1) would look like it was supported by a “trigger price” strategy. So trigger price strategies are consistent with a REBE, and hence with a rest point to the perfect synchronous game.

5.2.1 | Computational results when algorithms care about the future

Figure 5 provides price paths from AI algorithms that use perfect synchronous and asynchronous updating when $\beta = .95$, the state space is expanded to contain the prices each firm charged in the prior period, and demand is given by Equation (1).

As in Figure 2, which provided the comparison when $\beta = 0$, the asynchronous algorithm generates substantially higher price outcomes. Indeed setting $\beta = .95$ using a richer state space generates a distribution of pricing rest points from the asynchronous algorithm that stochastically dominates its analogue in the $\beta = 0$ case. However, the perfect synchronous algorithm now also converges to a price that is higher than the static Nash outcome. The median of the perfect synchronous price distribution is 6.25. This is substantially lower than the median price with asynchronous updating (which is 10), but substantially higher than the highest Nash price of 2.52. The rate of convergence for the perfect synchronous algorithm is still substantially faster than that of the asynchronous algorithm.⁴²

These results extend the conclusions of Calvano et al. (2020) to games played by AI algorithms that use perfect synchronous updating, but only if $\beta > 0$. Moreover, the theoretical discussion shows that when $\beta > 0$ the asynchronous algorithm can generate policy patterns that mimic those obtained from trigger-price strategies. However, a comparison among asynchronous algorithms, that is among the algorithm that use the learning rules used in Calvano et al. (2020), which we investigate further in Figure 6, throws a slightly different light on the problem.

This figure displays the frequency of realized prices generated by alternative asynchronous algorithms. Light gray bars are the static model (singleton state space and $\beta = 0$), medium gray bars are for an enriched state space

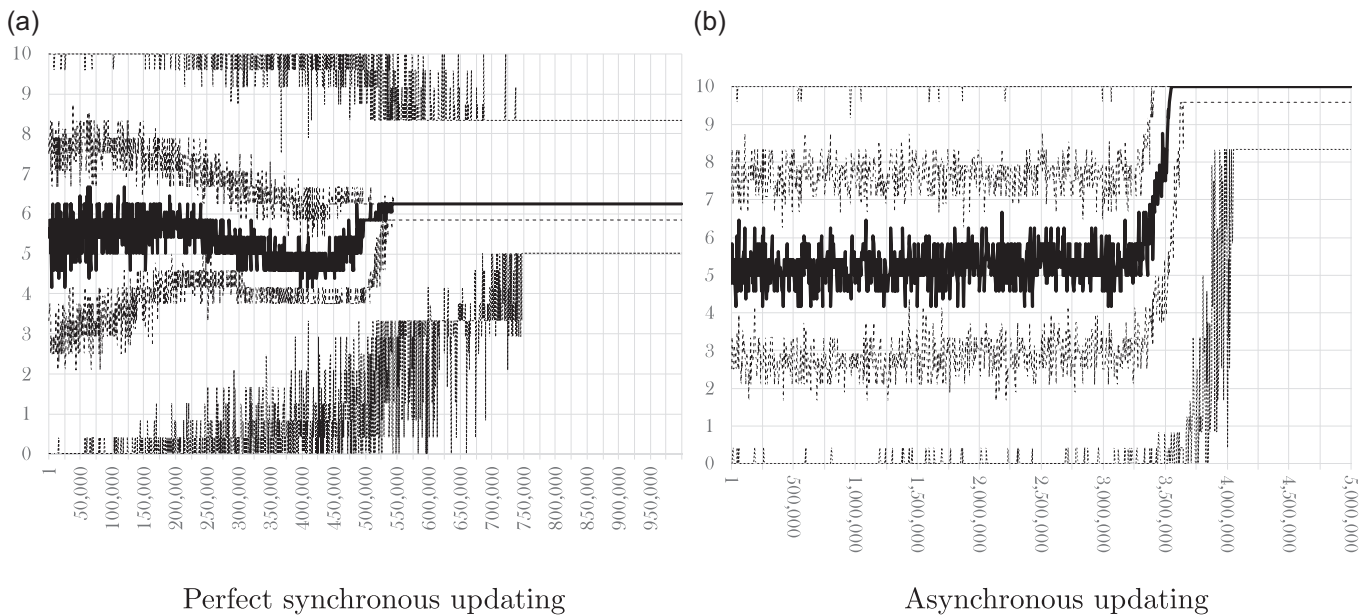


FIGURE 5 Price paths with richer state spaces and $\beta = .95$ (a) Perfect synchronous updating, (b) Asynchronous updating. Prices (vertical axis) by iteration (horizontal axis) from 100 simulations are shown. The lines, from bottom to top, represent the min, 25th percentile, median, 75th percentile, and max of the distribution of prices in each period. The discount factor, β , is set equal to .95. Results are shown for firm 1. The model is parametrized as follows. Feasible prices exist on a grid with 25 elements equally spaced between 0.1 and 10 inclusive. The state space contains the prices of both firms in the prior period. The weight on current returns in updating is given by $\alpha = .1$. Initial conditions are i.i.d. draws from $U[200, 210]$, for each $W(p)$ for each firm. In all other respects the model mirrors that in Figure 2.

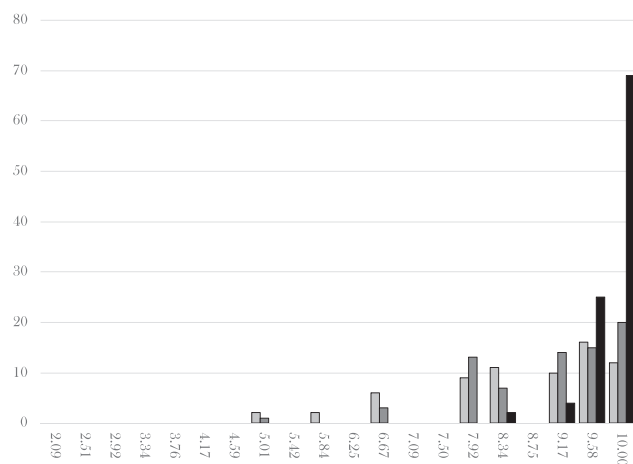


FIGURE 6 Price outcomes for models with asynchronous updating. The frequency of realized prices (horizontal axis), after the asynchronous algorithm has reached a resting point, is displayed. The model is the same as in Figure 5a, with the following adjustments. Light gray bars are the static model (singleton state space and $\beta = 0$). Medium gray bars are for an enriched state space (containing the price of each firm in the prior period) but with $\beta = 0$. Dark bars have the enriched state space and $\beta = .95$ as in Figure 5b. For each version of the model, results are constructed from 100 simulations.

(containing the price of each firm in the prior period) but with $\beta = 0$, and dark bars have the enriched state space and $\beta = .95$. As can be seen, there is little economically meaningful difference between the distribution of price outcomes generated by the static model (light gray) and that generated by the model augmented with an enriched state space (medium gray).⁴³ So a meaningful portion of any adverse price effect does not depend on leveraging history-dependent strategies.

More visually noticeable are the differences when we set $\beta = .95$ and we allow for history-dependent strategies; then the mean is 9.83 and the median is 10. The relative increase in the mass concentrated on a price of 10 is notable.

Keep in mind, however, that when $\beta = 0$ and we allow for history-dependent strategies, the median price outcome is 8.75. So the asynchronous algorithm generates a substantial increase in price even when it does not consider the impact of its play on the possibility of punishing in the future. That is, at least in our example, much of the difference

between rest point prices and Nash equilibrium prices generated by an asynchronous learning algorithm would exist even if we did not allow the algorithm to generate policies that mimic a punishment scheme.⁴⁴

5.2.2 | Extensions when $\beta > 0$

Table 3 reports the results of extensions to the basic specifications for perfect synchronous and asynchronous learning presented in Figure 5. For both, it begins by reporting results for the simulations underlying Figure 5. As noted when $\beta = .95$ and the state space includes the previous period's prices, both learning algorithms generate supra-competitive pricing outcomes with the distribution of prices generated by the asynchronous algorithm stochastically dominating that generated by the perfect synchronous algorithm.

The introduction of experimentation in this setting mirrors the qualitative results in Table 2. Experimentation appears to have a mild mitigating impact on price elevation in both cases. However, the effect is not as large as the effect of introducing a third firm in either case, with the difference being particularly noticeable for the perfect synchronous case. Perhaps most surprising is the impact on the asynchronous algorithm of imposing that the residual demand curve is decreasing in price (see Section 5.1). This has a greater downward impact on the price distribution, at all reported moments other than the minimum, than adding an extra firm.⁴⁵ With rich state spaces and $\beta > 0$, use of algorithms that abide by at least some information about the economic environment may be at least as important for pricing as increasing the number of firms in the market.

6 | OFFLINE VERSUS ONLINE PLAY

When setting a price in a new (or perturbed) environment there is a tradeoff between the perceived benefits of exploration and exploitation (choosing what is currently believed to be the most profitable, or “greedy” action). The choice made depends on the costs of the two actions. If the exploration is done “offline,” that is via computer simulation, the costs of exploration are near zero so exploring until convergence seems reasonable. However, exploration “online,” that is by setting prices in an actual market, can be costly. So it is unlikely that a firm would let the online algorithm run without supervision for the excessive number of iterations that seem to be required before convergence (see Figure 5).^{46,47}

In this section, we assume that the firm “trains” its algorithm offline, and then uses the converged policies from offline play as its initial perceptions for online play. Our goal is to throw light on whether we can expect converged policies to approximate online play.

The environment of the online game is likely to differ from that used in offline training as the online game will face uncertainty in both the competitor it faces and in features of the environment. So we proceed as follows. To allow for uncertainty in competitor's play we draw two different converged policies randomly from those underlying the results

TABLE 3 Pricing outcomes in extensions, when $\beta = .95$.

Algorithm	Treatment	Min	25th	Median	75th	Max
Perfect synchronous	$N = 2$ (as in Figure 5)	5.01	5.84	6.25	6.25	8.34
	$N = 2$, Experimentation ($\theta = 4$)	4.17	4.59	4.59	5.01	6.67
	$N = 3$	2.51	3.34	3.34	3.34	5.84
Asynchronous	$N = 2$ (as in Figure 5)	8.34	9.58	10.00	10.00	10.00
	$N = 2$, Experimentation ($\theta = 4$)	6.25	9.17	9.58	10.00	10.00
	$N = 3$	5.42	8.75	9.17	9.58	10.00
Synchronous using downward demand	$N = 2$	3.75	6.67	7.50	7.92	10.00

Note: For each treatment, 100 simulation runs were conducted. The minimum, 25th percentile, median, 75th percentile, and maximum of the prices for firm 1, across the 100 simulations, once all simulations have reached a rest point, is reported. The monopoly price is 10. Prices of 2.09 and 2.51 are both supportable as outcomes of a static Nash equilibrium. Experimentation occurs for the first 10 million iterations. The model is as per Figure 5, but for the noted changes in specification.

above for a given learning rule and play them against each other. To allow for uncertainty in the environment we do this once when demand is the same as it was in offline play and the initial condition is a Nash equilibrium, and once where demand is a random draw from a family with random slopes but monopoly price always 10 and a random initial condition. All of this is done using; perfect synchronous and asynchronous converged policies with and without online learning, and using perfect synchronous and asynchronous initial conditions (i.e., in online play $W_i^0(p)$ is set by the converged policies in offline play).⁴⁸

Table 4 summarizes results from repeating the different procedures 1000 times and computing the average price from the first 10,000 iterations of each. Panel A reports when we use the converged policies from runs with the new demand system and experimentation. The results are similar to those found in Table 3 for asynchronous learning (where converged prices varied between 6.25 and 10), though the new demand system generated notably lower prices than those in Table 3 for perfect synchronous learning (where they varied between 4.27 and 6.67).

Panel B reports results from playing two different converged policies against each other without learning. Both the perfect synchronous and asynchronous learning protocols lead to lower prices in the first 10,000 periods than those

TABLE 4 Online versus offline play.

	Within-Sim mean prices			# Prices observed		
	Mean price	Min	Max	Median	Max	% > 1
Panel A: Offline results						
Asynchronous	9.08	7.71	10.00	1	4	80
Perfect synchronous	3.91	3.71	4.86	1	2	4
Panel B: Online results without learning						
1. Asynchronous						
(initialized by offline asynchronous)						
Random demand, $s_i^0 = \text{random}$	4.71	2.00	10.00	9	15	99
Nonrandom demand, $s_i^0 = \langle 2, 2 \rangle$	4.73	2.00	10.00	9	15	100
2. Perfect synchronous						
(initialized by offline perfect synchronous)						
Random demand, $s_i^0 = \text{random}$	3.10	2.29	4.29	2	4	95
Nonrandom demand, $s_i^0 = \langle 2, 2 \rangle$	3.13	2.38	4.29	2	4	95
Panel C: Online results with learning (all initialized by offline asynchronous)						
3. Asynchronous						
Random demand, $s_i^0 = \text{random}$	4.76	2.97	10.00	15	15	100
Nonrandom demand, $s_i^0 = \langle 2, 2 \rangle$	6.29	4.54	10.00	15	15	100
4. Perfect synchronous						
Random demand, $s_i^0 = \text{random}$	4.62	2.62	9.12	15	15	100
Nonrandom demand, $s_i^0 = \langle 2, 2 \rangle$	4.71	4.45	10.00	15	15	100
Panel D: Online results with learning (all initialized by offline perfect synchronous)						
5. Asynchronous						
Random demand, $s_i^0 = \text{random}$	4.31	2.54	9.88	15	15	100
Nonrandom demand, $s_i^0 = \langle 2, 2 \rangle$	3.99	2.95	9.09	5	15	90
6. Perfect synchronous						
Random demand, $s_i^0 = \text{random}$	3.94	2.57	4.46	15	15	100
Nonrandom demand, $s_i^0 = \langle 2, 2 \rangle$	3.66	3.18	4.29	3	15	90

Note: Prices are market clearing prices ($\min\{p_1, p_2\}$). 1000 online simulations were run, in which two AIs were randomly selected from the 100 offline simulations, matched, and played against each other for 10,000 periods. See footnote 6 for other details.

FIGURE 7 The algorithm coordination game. The payoffs reflect the pattern of expected payoffs reported in Figure 2 and Appendix A.5.

		Firm Y	
		<i>Asynch.</i>	<i>Sync.</i>
Firm X	<i>Asynch.</i>	(8.12, 8.12)	(2.13, 2.13)
	<i>Sync.</i>	(2.13, 2.13)	(2.13, 2.13)

reported in Panel A, but the drop in the asynchronous mean price is larger, as it is reduced by half, while the perfect synchronous mean is 20% lower. This, combined with the fact that the mean price of the asynchronous algorithm in Panel A is so much higher than that of the synchronous algorithm, means that the absolute magnitude of the difference in the online means from the different learning rules, though still notable, falls dramatically (from 5.2 to 1.4). Randomization of demand and initial conditions does not seem to matter.

Panels C and D report online results with online learning. We begin by comparing rows 3–rows 6; that is asynchronous learning with asynchronous initial conditions to perfect synchronous learning with a perfect synchronous initial condition. Allowing for online learning generates higher mean prices for both algorithms, but the difference is more noticeable in the asynchronous algorithm when demand and initial conditions are random, while it is more noticeable for the synchronous algorithm when demand and initial conditions are fixed. However the qualitative comparison to Panel A (to those from offline learning) do not change from the online case without learning; (i) both mean prices fall, (ii) the asynchronous mean remains higher than the synchronous mean, and (iii) the difference between them falls (to 0.82 with random demand and initial conditions and to 2.57 without).

Rows 3 and 4 compare asynchronous to perfect synchronous online models when both start with an asynchronous initial condition, while comparing lines 5–6 make the same comparison when starting with a perfect synchronous initial condition. The three qualitative results noted above carry over to these comparisons though, as one might expect, using the same initial conditions causes the differences between prices to fall further.

There are a number of other ways that the prices generated from the online protocols differ from those that result from an offline training process. First, as the last three columns of Table 4 show, many more distinct prices are observed in online play than with the play generated by converged offline training policies. Second, in all online cases at least one of the firms' prices cycle, and the cycles do not die out in the 10,000 iterations. We also found cycles in some offline cases in the environment used here, but then both firms played the same price during the cycle. In the online cases the two firms played different prices.

Our takeaway from the comparisons of our offline to our online results is a word of caution. We should be careful in viewing the limiting policies from offline play as a good approximation to what AI pricing algorithms would actually generate were they used online.

7 | CONCLUSION

The results in this paper relate to contemporary policy discussions about appropriate competition policy in a world in which decisions are delegated to algorithms. We illustrate the impact of the design on equilibrium prices. Much of this policy discussion has centered on traditional antitrust tools, particularly laws prohibiting collusion.⁴⁹ The results here present significant challenges for the application of those traditional tools. We show the potential for algorithms to lead to supra-competitive pricing in settings where, as in Harrington (2018), it cannot match any economic definition of collusive behavior.

We have not explicitly discussed the choice of algorithms. Of course, even if a firm understands that choosing a naive (e.g., asynchronous) algorithm will lead to supra-competitive pricing and, as a consequence, delegates its pricing to such an algorithm, this is distinct from doing so in concert with other firms as part of some commitment or agreement. Figure 7 below shows the pattern of average (rest point) payoffs arising from alternative choices of algorithmic designs given the offline results reported in this paper for the static Bertrand model. It is a Nash equilibrium for both firms to choose to implement asynchronous algorithms. Indeed, no matter what a rival firm implements, each firm is weakly better off with the asynchronous algorithm. So employing an asynchronous algorithm is a weakly dominant strategy, though as noted in Table 4, the actual price effects may be more muted in online play.⁵⁰

That the choice of algorithms leading to supra-competitive pricing could be an equilibrium outcome in a static game, seems at odds with the requirement that illegal cartels have some form of agreement at their core. This conclusion resonates with ongoing expressions of discomfort, on the part of antitrust scholars, with the heavy emphasis

that cartel enforcement puts on finding an agreement (see for instance, Turner (1962), Posner (1976), Whinston (2006), Kaplow (2013), and, in the context of algorithms specifically, Harrington (2018).

Whether through regulation, or through antitrust oversight, the question of how to identify a “procompetitive” implementation of an algorithm seems to be central to any evaluation exercise that a regulatory agency might conduct. Most likely this requires access to the underlying code.⁵¹ At least the results in this paper point toward procompetitive implementations being those that are informed by an understanding of the wider economic environment and that incorporate counterfactual alternatives to the play actually engaged in when learning. Thus, indicia of the possible existence of a “procompetitive” implementation may include the incorporation of a demand model, likely informed by statistical or econometric studies (such as A–B testing) to generate returns to alternative actions. Inducing the algorithm to respond to changes in its environment may also lead to useful indicators of “procompetitiveness” of the algorithm.⁵² Additionally, the greater the number of firms in a market, the more muted any supra-competitive pricing is likely to be.⁵³

Developing an empirical understanding of how algorithm adoption impacts pricing, and the choices the algorithms that are actually used gives management, seems invaluable. Empirical studies of the nature and the impact of AI pricing algorithms that have been adopted are one way of evaluating the relative merits of human and algorithmic control of pricing in markets. Assad et al. (2020) provides a valuable first step in this direction. Additional research in this vein, guided by the growing theoretical and computational work on algorithms, strikes us as particularly valuable.

ACKNOWLEDGMENTS

An earlier version of this paper was circulated under the title, “Artificial Intelligence and Pricing: The Impact of Algorithm Design” and as NBER Working Paper 28535. We would like to thank participants in various seminars and conferences for their comments. Excellent research assistance was provided by Lorenzo Cattivelli, Masha Melkonyan, and Kylie Toh.

DATA AVAILABILITY STATEMENT

The data and code that support the findings of this study are available from the corresponding author upon request. The base code is found in the appendix and has been used for replication by other researchers.

ORCID

John Asker  <http://orcid.org/0000-0003-3371-6497>

ENDNOTES

- ¹ See, for instance, the discussions in Chen et al. (2016), Competition and Markets Authority (2018), Derakhshan et al. (2016), Brown and MacKay (2021), Assad et al. (2020), and Calvano et al. (2020).
- ² See, for instance, Sutton and Barto (2018).
- ³ As discussed later, there are a wide range of ways to implement this broad-brush approach. An early, seminal discussion can be found in Watkins and Dayan (1992). Sutton and Barto (2018) provide a contemporary treatment.
- ⁴ Quite literally, the information that is leveraged is the action (charging \$7) and the realized profit from charging \$7. No other information is used in updating the values attached to actions (in the learning process).
- ⁵ We call an updating protocol *synchronous* if it allows the AI to update the return from playing prices other than the price actually paid. *Perfect synchronous* updating assumes that the AI can compute these counterfactuals perfectly (as in the example in the preceding paragraph). Other implementations, like the *synchronous using downward demand* version discussed in this paper, may use approximations or bounds to guide updating.
- ⁶ The policy and legal debates are moving faster than the economic literature. On the policy side see Competition and Markets Authority (2018), OECD (2017), Sims (2017), and Federal Trade Commission (2018a). All these references discuss algorithms, and AI in particular, as potentially facilitating collusive outcomes. Margrethe Vestager, the EU Commissioner for Competition, commented in 2018 that “The challenges that automated systems create are very real ... If they help companies to fix prices, they really could make our economy work less well for everyone else” (quoted in Hirst, 2018). For commentary see Harrington (2018), Schwalbe (2018), Assad et al. (2021), and Veljanovski (2022).
- ⁷ This is often referred to as “Q-learning.” However, in the machine learning (or AI) literature, Q-Learning tends to have a broader meaning that subsumes both asynchronous and synchronous learning. See, for instance, Watkins and Dayan (1992).
- ⁸ An important part of an AI algorithm is the exploration procedure that it adopts. One approach is that every period the player chooses, with some small probability, a suboptimal action and then uses the outcome to update the value for this action. Another option is to use

an exploration method called “optimistic initial values” in which the initial values are set at a high level encouraging the player to explore these values. For a discussion of these alternative approaches see Calvano et al. (2021a). In our initial results (in Figure 1, for instance) we use the optimistic initial value approach. In Section 5.1 we introduce also random experimentation and discuss the implications of the two procedures on the algorithmic outcome.

- ⁹ Appendix A.4, conducting the same exercise on quantity-setting (Cournot) games, replicates known results in Waltman and Kaymak (2008) and shows that our distinction between perfect synchronous and asynchronous learning is also relevant in that setting.
- ¹⁰ Q-learning algorithms that employ experimentation learn about the values of choosing nonoptimal actions by using an ϵ -greedy algorithm. The algorithm chooses with probability $1 - \epsilon$ the optimal action, and with probability ϵ it experiments and chooses randomly from all feasible actions. Once it chooses an action it may update the value of taking that action. See Sutton and Barto (2018). Calvano et al. (2020) provide an application in the economics literature.
- ¹¹ There may also be a cost in terms of the revenue generated, as when the algorithm experiments it does not use what it currently views as the optimal policy.
- ¹² Johnson et al. (2020) engage in a related computational study of platform competition, showing that platforms can design their marketplaces in a way to limit the prospects of collusion by merchants, and that this can be successful even when Q-learning algorithms are used by merchants. They show that platform itself can benefit from such design policies, in addition to raising consumer surplus.
- ¹³ Waltman and Kaymak (2008) conduct a related computational experiment in a static Cournot setting. Our paper differs in investigating the impact of the sophistication of the learning process, and comparing environments with static and repeated interactions. A computer science literature also considers the role of AIs using reinforcement learning in shaping market outcomes (see, for instance, Tesauro & Kephart, 2002 and Sandholm & Crites, 1995). Ongoing work in this computer science literature has focused on designing algorithms that can sustain cooperation in repeated prisoner dilemmas (see, for instance, Xue et al., 2018). To our knowledge, this computer science literature has not explored the sensitivity of market outcomes to the basic informational requirements of the AIs learning protocol, or how that interacts with other features of a richer pricing game environment. Mnih et al. (2015) compares the broader state of the AI research in computer science to human learning, and considers an extension to the AI literature.
- ¹⁴ Aparicio et al. (2021) provide related evidence in the context of supermarkets. Similarly, Normann and Sternberg (2021) provide related experimental evidence.
- ¹⁵ In the Bertrand game case, this would converge (under some standard conditions) to the Nash equilibrium of the game.
- ¹⁶ Another point of distinction is that, in our setting, players do not “learn” about the probability distribution of strategies chosen by other players. Rather, what they have in memory is only the value associated with every action.
- ¹⁷ Examples of contributions directed at this policy debate include Mehra (2015), Ezrachi and Stucke (2017), Kühn and Tadelis (2017), Schwalbe (2018), de Corniere and Taylor (2020), Assad et al. (2021), and Veljanovski (2022). Goldfarb et al. (2019) provide a broader overview of the likely impact of AI on the economy at large.
- ¹⁸ We assume such a structure purely for computational convenience. Any function where residual demand is decreasing in own price would suffice for our major results.
- ¹⁹ For the moment, we ignore the possibility that repetition may enable history-dependent strategies.
- ²⁰ This equilibrium structure is purely a consequence of the set of feasible prices being discrete. If prices were continuous, this would reduce to the familiar undergraduate example in which $p = c$ is the equilibrium.
- ²¹ We use a variant of Q-learning which was introduced by Watkins (1989). For a survey of the different methods of implementing reinforced learning algorithms, see Sutton and Barto (2018).
- ²² Pakes and McGuire (2001) show how to use reinforcement learning to compute Markov Perfect solutions to dynamic games, Fershtman and Pakes (2012) and Asker et al. (2020) provide their relationship to EBE (a notion of equilibrium used below), and Calvano et al. (2020) use them to study pricing in markets where prices are set by computer algorithms. Igami (2020) discusses the many links between AIs and applied econometric practice, particularly in the estimation of dynamic models.
- ²³ We make this assumption to ensure that at least some subset of the state space is visited repeatedly (i.e., “infinitely often”). This will allow us to formally examine limits of the price process.
- ²⁴ The influence of the state space on the set of feasible policies mirrors the influence of the state space on the types of strategies that can be played in a dynamic game. It is common to restrict state spaces in dynamic games to only include payoff-relevant and informationally relevant variables (see Asker et al., 2020; Fershtman & Pakes, 2012; and Maskin & Tirole, 2001). For the purposes of this paper, the only necessary restriction is that the state space be finite.
- ²⁵ For an analysis of an AI pricing algorithm that takes this explicitly into account see Brown and MacKay (2021).
- ²⁶ The terms synchronous and asynchronous are slight abuses of language in the context of the broader reinforcement learning literature. In that literature “asynchronous” is used to refer to any algorithm in which only a subset of the $W_i(p|s)$'s in the system are updated at each iteration, and that subset can vary with s (see Sutton & Barto, 2018).

- 27 That is, there are two ways to induce an algorithm to explore. One is through relatively large initial perceptions of different prices; a high $\{W^0(p|s)\}_{p \in P, s \in S}$. Another is through experimentation. Below we augment the algorithm to allow for experimentation, and hence both kinds of exploration.
- 28 The full details of the parametrization are found in the notes at the bottom of the figure.
- 29 Most paths settle on a single price by iteration 2500.
- 30 We assume for convenience that there is always a single best response. A best response will always exist since the set of prices is finite.
- 31 For simplicity, here and below we are assuming that the (s_i, s_j) associated with the rest point (p_i^*, p_j^*) is unique.
- 32 Formally, points in $\mathcal{R}$ that can only transit to other points in $\mathcal{R}$ no matter which feasible policy is played are referred to as “interior points,” while points in $\mathcal{R}$ for which there is a feasible policy which would transit to points outside of $\mathcal{R}$ are referred to as “boundary points.”
- 33 If we simply bin prices and do not impose further restrictions, the memory requirements will increase exponentially in the number of firms, though if we assume, as is often done in applied work, that the policies are exchangeable in the states of competitors, the rate of growth in the number of competitors decreases, from exponential to geometric, see Pakes and McGuire (1994).
- 34 For $N \in \{2, 3, 4\}$ the highest static Nash price is 2.13 (2.03 is also a Nash outcome). For $N \geq 5$ the only static Nash equilibrium price is 2.03.
- 35 These results are qualitatively similar in an environment with a Logit demand system in Appendix A.2. Interestingly, in those results the asynchronous case is shown to reach results higher than monopoly, and also below Nash, in some instances.
- 36 For $N = 6$, the percentage greater than static Nash is 2.1. In the case of the Logit model reported in Appendix A.2, the percentages for $N = \{4, 5, 6\}$ are 71.9, 66.6, and 60.6, respectively, suggesting that the results reported in Table 1 may understate these probabilities for, at least some, alternative environments.
- 37 The addition of experimentation to the perfect synchronous algorithm (Figure 2a) would be redundant as the perfect synchronous algorithm updates all possible actions anyway.
- 38 See Sutton and Barto (2018).
- 39 These rules are for $p \geq c$. For $p < c$ these are reversed.
- 40 Appendix A.2 shows the impact of this augmented procedure in an environment with Logit demand. Mitigation still occurs, but is somewhat muted relative to the homogenous Bertrand example.
- 41 Examples might include: using the realizations of price and quantity to approximate demand, using one or more misspecified demand models, or using local regression approaches to approximate W 's that are close to those that can be updated precisely (see Farias et al., 2012 for a related approach in the context of computing approximate MPE in dynamic oligopoly games).
- 42 The perfect synchronous price distribution stabilizes after about 750,000 iterations, while asynchronous price distribution does not stabilize until around 4,000,000 iterations.
- 43 The mean (and median) prices in the “static” (light gray) and “ $\beta = 0$ ” (medium gray) cases are 8.33 (8.34) and 8.73 (8.75), respectively.
- 44 Appendix A.3 illustrates two examples of the path of play following an optimal deviation from a rest point reached by the asynchronous algorithm.
- 45 Leveraging this tenet would have no additional benefit for the perfect synchronous algorithm.
- 46 To give a sense of this, if an iteration of play is 30 s long, 1 million iterations is approximately a year of play. It seems implausible that a firm would delegate unsupervised play to an AI for a year and commit to the potential costs of a mistake for such a long period. So we expect that in most applications some of the learning occurs offline. This point is also made by Calvano et al. (2020) and is well understood in the computer science literature and among industry practitioners.
- 47 Since the accuracy of the policy evaluations at a state depend on the number of times states are visited, it may also be in a firm's interest to coarsen the state space in online games and increase precision at the cost of incurring some bias. We do not investigate this tradeoff here.
- 48 This new environment is parameterized so that the monopoly price is always 10 and marginal costs (c) are equal to 2. There are 15 feasible prices equally spaced between 2, and 10 inclusive. The static Nash equilibria are at 2 and 2.57. We set $D(p) = 1 + 18 \cdot \tan(\theta) - \tan(\theta)p$ if $p \leq 10$, and zero otherwise. θ is the angle between the y-axis (price-axis) and an extension of the demand curve through the y-axis (price-axis). This specification allows for the possibility of rotations in the demand curve (via the choice of θ), which changes incentives to offer alternative prices, while ensuring that 10 is always the monopoly price. When we say that demand is randomized in online play, θ is an i.i.d draw from $U(0, 90)$ for each market (θ does not vary across periods of play). In offline play, or in online play where demand is not randomized, total demand at any given price p is equal to $E_\theta D(p)$. The state space contains the firm's price in the prior iteration and the lowest price offered by its rivals (when two firms are present in the market this amounts to the state space being both firm's prior prices). Each firm sets $\beta = 0.95$. The weight on current returns in updating is given by $\alpha = 0.1$.

In offline play, Initial conditions are set such that each $W_i^0(p)$ is an i.i.d. draw from $U[0, 10] + 5000$. This value is higher than the maximum possible W for any action, and is set to encourage exploration. Experimentation is always used in the offline play. Experimentation occurs with frequency $e^{-\frac{k}{10^5}}$. After the 460-thousandth iteration the probability of experimentation is approximately 1 in 100. By the 5 millionth iteration the probability of experimentation is less than 1 in 1 billion. Experimentation is not used in online play.

⁴⁹ See, for instance, the discussion in Harrington (2018).

⁵⁰ This is also true if we add downward sloping demand to the set of algorithms that can be chosen. In the Logit version of the model discussed in the appendix, employing the asynchronous algorithm is a strictly dominant strategy.

⁵¹ Alternatively, one might induce the program to respond to various inputs and use the output to infer about the algorithm that generated it. This may be necessary if the program can be used to generate pricing recommendations, but access to underlying code is restricted.

⁵² Here the changes should be small enough to have no meaningful impact on the pricing of any competing firm, and exist merely to inform the computation of counterfactual returns. If experimentation occurs on subpopulations that are large, then the possibility may arise that experimentation distorts algorithmic learning.

⁵³ Additionally, conditional on the design of the algorithm, the more algorithms are trained to optimize current rather than future profits, the better for competition. Similarly, the less information about competitors' actions that resides in the state space, the better.

⁵⁴ Differences in time taken to reach a rest point are also qualitatively similar. For $N = 2$, it takes 2910 iterations to reach the reported distribution in the asynchronous case, and 100 iterations in the perfect synchronous case.

⁵⁵ Differences in time taken to reach a rest point are also qualitatively similar. It takes 6706 iterations to reach the reported distribution in the asynchronous case, and 346 iterations in the perfect synchronous case.

⁵⁶ It takes 950 iterations to reach the reported distribution

REFERENCES

- Agrawal, A., Gans, J., & Goldfarb, A. (2019). *The economics of artificial intelligence: An agenda*. University of Chicago Press.
- Aparicio, D., Metzman, Z., & Rigobon, R. (2021). *The pricing strategies of online grocery retailers*. NBER WP 28639.
- Asker, J., Fershtman, C., Jeon, J., & Pakes, A. (2020). A computational framework for analyzing dynamic auctions: The market impact of information sharing. *RAND Journal of Economics*, 51(3), 805–839.
- Asker, J., Fershtman, C., & Pakes, A. (2022). Artificial intelligence, algorithm design and pricing. *AEA Papers and Proceedings*, 112, 452–56.
- Assad, S., Clark, R., Ershov, D., & Xu, L. (2020). *Algorithmic pricing and competition: Empirical evidence from the German retail gasoline market*. Working paper.
- Assad, S., Calvano, E., Calzolari, G., Clark, R., Denicolo, V., Ershov, D., Johnson, J., Pastorello, S., Rhodes, A., Xu, L., & Wildenbeest, M. (2021). Autonomous algorithmic collusion: Economic research and policy implications. *Oxford Review of Economic Policy*, 37, 459–478.
- den Boer, A., Meylahn, J., & Schinkel, M. (2022). *Artificial collusion: Examining supracompetitive pricing by Q-learning algorithms*. Mimeo.
- Brown, Z., & MacKay, A. (2021). Competition in pricing algorithms. *American Economic Journals: Microeconomics* (forthcoming).
- Calvano, E., Calzolari, G., Denicolò, V., & Pastorello, S. (2020). Artificial intelligence, algorithmic pricing and collusion. *American Economic Review*, 110(10), 3267–3297.
- Calvano, E., Calzolari, G., Denicolò, V., & Pastorello, S. (2021a). Algorithmic collusion with imperfect monitoring. *International Journal of Industrial Organization*, 79, 102712.
- Calvano, E., Calzolari, G., Denicolò, V., & Pastorello, S. (2021b). *Algorithmic collusion, genuine and spurious*. CEPR Discussion paper no. 16393.
- Chen, L., Mislove, A., & Wilson, C. (2016). An empirical analysis of algorithmic pricing on amazon marketplace. In *Proceedings of the 25th International Conference on World Wide Web* (pp. 1339–1349).
- Competition and Markets Authority. (2018). *Pricing algorithms: Economic working paper on the use of algorithms to facilitate collusion and personalised pricing*. CMA Working Paper CMA94.
- de Corniere, A., & Taylor, G. (2020). *Data and competition: A general framework with applications to mergers, market structure, and privacy Policy*. TSE Working Paper no. 20–1076.
- Derakhshan, A., Hammer, F., & Demazeau, Y. (2016). PriceCast fuel: Agent based fuel pricing. advances in practical applications of scalable multi-agent systems. the PAAMS collection. *Lecture Notes in Computer Science*, 9662, 247–250.
- Epivent, A., & Lambin, X. (2022). *On algorithmic collusion and reward-punishment schemes*. Mimeo.
- Erev, I., & Haruvy, E. (2016). Learning and the economics of small decisions. *The Handbook of Experimental Economics*, 2(10), 638–716.
- Eschenbaum, N., Mellgren, F., & Zahn, P. (2022). *Robust algorithmic collusion*. Mimeo.
- European Commission. (2020). *Inception impact assessment: New competition tool*.
- Ezrachi, A., & Stucke, M. E. (2017). Artificial intelligence & collusion: When computers inhibit competition. *University of Illinois Law Review*, 2017, 1775–1810.
- Farias, V., Saure, D., & Weintraub, G. Y. (2012). An approximate dynamic programming approach to solving dynamic oligopoly models. *Rand Journal of Economics*, 43(2), 253–282.

- Federal Trade Commission. (2018a). *FTC hearing #7: The competition and consumer protection issues of algorithms, artificial intelligence, and predictive analytics*.
- Federal Trade Commission. (2018b). *FTC hearing 7: Nov. 14 welcome remarks and session 1 algorithmic collusion*.
- Fershtman, C., & Pakes, A. (2012). Dynamic games with asymmetric information: A framework for empirical work. *Quarterly Journal of Economics*, 127(4), 1611–1661.
- Fudenberg, D., & Levine, D. K. (2016). Whither game theory? Towards a theory of learning in games. *Journal of Economic Perspectives*, 30(4), 70–151.
- Goldfarb, A., Gans, J., & Agrawal, A. (2019). *The economics of artificial intelligence: An agenda*. University of Chicago Press.
- Hansen, K., Misra, K., & Pai, M. (2020). *Algorithmic collusion: Supra-competitive prices via independent algorithms*. CEPR Discussion Paper, no. DP14372.
- Harrington, J. (2018). Developing competition law for collusion by autonomous artificial agents. *Journal of Competition Law and Economics*, 14, 331–363.
- Hart, S., & Mas-Colell, A. (2003). Uncoupled dynamics do not lead to Nash equilibrium. *American Economic Review*, 93(5), 1830–1836.
- Hirst, N. (2018). *When Margrethe Vestager takes antitrust battle to robots*. Politico.
- Igami, M. (2020). Artificial intelligence as structural estimation: Deep Blue, Bonanza, and AlphaGo. *Econometrics Journal*, 23(3), S1–S24.
- Johnson, J., Rhodes, A., & Wildenbeest, M. (2020). *Platform design when sellers use pricing algorithms*. Working Paper.
- Kaplow, L. (2013). *Competition policy and price fixing*. Princeton University Press.
- Klein, T. (2019). *Autonomous algorithmic collusion: Q-learning under sequential pricing*. Amsterdam Law School Research Paper no. 2018–15.
- Kühn, K., & Tadelis, S. (2017). *The economics of algorithmic pricing: Is collusion really inevitable*. Unpublished Manuscript.
- Liang, A. (2020). *Games of incomplete information played by statisticians*. Mimeo.
- Leisten, M. (2021). *Algorithmic competition, with humans*. Mimeo.
- Martin, S., & Rasch, A. (2022). *Collusion by algorithm: The role of unobserved actions*. Mimeo.
- Maskin, E., & Tirole, J. (2001). Markov perfect equilibrium: I. Observable actions. *Journal of Economic Theory*, 100(2), 191–219.
- Maskin, E., & Tirole, J. (1988). A theory of dynamic oligopoly, II: Price competition, kinked demand curve, and Edgeworth cycles. *Econometrica*, 56(3), 571–599.
- Mehra, S. (2015). Antitrust and the robo-seller: Competition in the time of algorithm. *Minnesota Law Review*, 100, 1323–1375.
- Miklós-Thal, J., & Tucker, C. (2019). Collusion by algorithm: Does better demand prediction facilitate coordination between sellers? *Management Science*, 65(4), 1552–1561.
- Milgrom, P., & Roberts, J. (1991). Adaptive and sophisticated learning in normal form games. *Games and Economic Behavior*, 3(1), 82–100.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A., Veness, J., Bellemare, M., Graves, A., Riedmiller, M., Fidjeland, A., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518, 529–533.
- Normann, H.-T., & Sternberg, M. (2021). *Hybrid collusion: Algorithmic pricing in human-computer laboratory markets*. Mimeo.
- O'Connor, J., & Wilson, N. (2019). *Reduced demand uncertainty and the sustainability of collusion: How AI could affect competition*. Working Paper.
- OECD. (2017). *Algorithms and collusion: Competition policy in the digital age*.
- Pakes, A., & McGuire, P. (1994). Computing Markov-perfect Nash equilibria: Numerical implications of a dynamic differentiated product model. *The RAND Journal of Economics*, 25(4), 555–589.
- Pakes, A., & McGuire, P. (2001). Stochastic algorithms, symmetric Markov perfect equilibrium, and the ‘curse’ of dimensionality. *Econometrica*, 69(5), 1261–1281.
- Posner, R. (1976). *Antitrust law: An economic perspective*. University of Chicago Press.
- Salcedo, B. (2015). *Pricing algorithms and tacit collusion*. Working Paper.
- Sandholm, T., & Crites, R. (1995). On multiagent Q-learning in a semi-competitive domain. In *International Joint Conference on Artificial Intelligence* (pp. 191–205).
- Schwalbe, U. (2018). Algorithms, machine learning, and collusion. *Journal of Competition Law and Economics*, 14(4), 568–607.
- Sims, R. (2017). The ACCC's approach to colluding robots. The Australian Competition and Consumer Commission. Available at <https://www.accc.gov.au/speech/the-accc%E2%80%99s-approach-to-colluding-robots-address>
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction*. MIT Press.
- Tesauro, G., & Kephart, J. O. (2002). Pricing in agent economies using multi-agent Q-learning. *Autonomous Agents and Multi-Agent Systems*, 5(3), 289–304.
- Turner, D. (1962). The definition of agreement under the sherman act: Conscious parallelism and refusals to deal. *Harvard Law Review*, 75, 655–706.
- Veljanovski, C. (2022). Algorithmic antitrust: A critical overview. In A. Portuese (ed.), *Economic Analysis of Law in European Legal Scholarship* (pp. 39–64). Springer.
- Waltman, L., & Kaymak, U. (2008). Q-learning agents in a Cournot oligopoly model. *Journal of Economic Dynamics and Control*, 32(10), 3275–3293.
- Watkins, C. (1989). *Learning from delayed rewards* [PhD Thesis], University of Cambridge, England.
- Watkins, C., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8, 279–292.

Whinston, M. (2006). *Lectures on antitrust economics*. MIT Press.

Xue, L., Sun, C., Wunsch, D., Zhou, Y., & Yu, F. (2018). An adaptive strategy via reinforcement learning for the prisoner's dilemma game. *Journal of Automatica Sinica*, 5(1), 301–310.

How to cite this article: Asker, J., Fershtman, C., & Pakes, A. (2023). The impact of artificial intelligence design on pricing. *Journal of Economics & Management Strategy*, 1–29. <https://doi.org/10.1111/jems.12516>

APPENDIX A

A.1 | Code for the baseline

The code below, comprises the core content of the m-file required to run the baseline model in Matlab. It is included here to assist the reader in understanding implementation. The code was run in Matlab version R2015a.

```
%-----Set parameters-----%
theta=10; D=1; nfirms=2; c=2; alpha=.1; beta=0;
pL=0.01; pH=theta; dim=100; p=(linspace(pL,pH,dim));
%-----Set updating protocol-----%
synchronous=0;
%-----Initialization-----%
rng(2); W=[]; W(:,1,1)=10+10*rand(dim,1); W(:,1,2)=10+10*rand(dim,1);
itermax=10000; h=[]; pstar=[]; k=1;
%-----Loop through learning periods-----%
while k<=itermax
    %-----determine price chosen-----%
    for i=1:nfirms
        [piestar2,m]=max(W(:,k,i)); pstar(i,k)=p(m,:);
    end
    %-----Updating-----%
    for i=1:nfirms
        pstaro(i,k)=pstar(i,k); pstar(i,k)=pH+1;
        prival=min(pstar(:,k)); nshares=sum(pstar(:,k)==prival)+1;
        less=find(p<prival);
        equal=find(p==prival);
        above=find(p>prival);
        W_all(less,k+1,i)=D*(p(less)c);
        W_all(equal,k+1,i)=(1/nshares)*D*(p(equal)c);
        W_all(above,k+1,i)=0*(p(above)c);
        W(:,k+1,i)=W_all(:,k+1,i);
        if synchronous==0
            W(:,k+1,i)=W(:,k,i);
            m=find(pstaro(i,k)==p);
            W(m,k+1,i)=W_all(m,k+1,i);
        end
        W(:,k+1,i)=W(:,k+1,i)*alpha+W(:,k,i)*(1-alpha);
        pstar(i,k)=pstaro(i,k);
    end
    k=k+1;
end
```

This code is used as the base for producing all the figures. For instance, Figure 2 uses this base code, and loops over 100 simulation iterations, with a new set of initial condition draws in each iteration.

A.2 | Results for a Logit demand system

This section reports results reproducing the core static computational analysis in the main text, but substituting the perfect substitutes demand system for a logit style demand system in which each firm's product is differentiated. As in the model considered in the main text, firms play a Bertrand pricing game. The form of the demand system is

$$Q_i = \frac{e^{a-bp_i}}{1 + e^{a-bp_i} + \sum_{j \neq i} e^{a-bp_j}}, \quad (\text{A1})$$

where Q_i is the quantity demanded of firm i 's product. The parameters are set such that $a = 40$ and $b = 4$. In all other respects, the model and parametrization (including the selection of initial values) are unchanged from that underlying Figure 2.

Table A1 shows the distribution of rest points reached in this new pricing game as the number of firms varies. It shows that the core results are qualitatively unchanged in the alternative Logit demand environment.⁵⁴

A.3 | Policies in the repeated game with asynchronous learning

Figure A1 provides examples of the path of play, for two simulations, with different initial conditions, in the repeated game with asynchronous learning. The path of play is at the rest point in the first 20 periods ($p = 10$). At period 21, firm 1 drops

TABLE A1 Pricing outcomes with Logit demand.

Algorithm	N	Minimum	25th Percentile	Median	75th Percentile	Max
Perfect synchronous	2	2.53	2.53	2.53	2.53	2.53
	3	2.33	2.43	2.43	2.43	2.43
	5	2.33	2.33	2.33	2.33	2.33
	10	2.23	2.23	2.33	2.33	2.33
Asynchronous	2	6.17	8.13	8.59	9.19	10.00
	3	2.63	4.95	6.37	7.93	9.80
	5	2.23	2.43	2.43	2.63	9.90
	10	2.13	2.33	2.43	2.43	2.73
Asynchronous w. downward demand	2	2.33	2.63	2.89	3.59	9.60

Note: For each N , 100 simulation runs were conducted. The distribution of prices for firm 1, once all simulations have reached a rest point, is reported. The model is as per Figure 2, but for the substitution of the Logit demand system described in equation A1 and the reported changes in N . "downward demand" specifications mirror those discussed in Section 5.1.

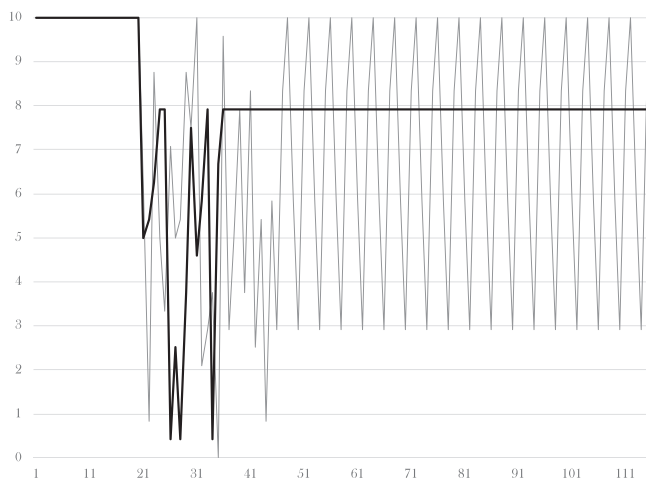


FIGURE A1 Price paths following an optimal deviation. The underlying model is that in Figure 5b.

the price by an increment (this perturbation is imposed exogenously). Firm 2 responds according to the policies in memory (the AI does not do any learning in this sequence of play). Figure A1 shows the path of play of firm 2, in each of the two simulations selected as examples. As can be seen, both paths bounce around a lot for the 20–30 periods following the deviation and then settle down. One settles down at a constant price, while the other settles into a regular oscillating pattern.

A.4 | Results for a Cournot game

This section reports results reproducing the basic core static computational analysis in the main text, but replacing the Bertrand game with a homogenous good Cournot game. The form of the demand system used in this new game form is

$$Q = a - P, \quad (\text{A2})$$

where $Q = \sum_i q_i$, q_i is the chosen output of firm i and P is the market price. The model is parameterized such that the demand intercept, a , is equal to 10. Quantities (q_i 's) can take on 150 values evenly spaced between 1.51 and 3, inclusive. Initial conditions are set such that each initial $W(q)$ is an independent draw from $U[25, 35]$. In all other respects the model and parametrization are unchanged from that underlying Figure 2.

Table A2 shows the distribution of rest points reached in this new pricing game for $N = 2$. It shows that the core results are qualitatively unchanged in the Cournot environment, illustrating that the patterns are not specific to games of strategic complements. The same patterns can be seen in games of strategic substitutes.⁵⁵

A.5 | Results when one firm learns with an asynchronous algorithm and one with a perfect synchronous algorithm

This section reports results for the model underlying Figure 2, with the adjustment that one firm's algorithm uses perfect synchronous updating, while the other uses asynchronous updating. Table A3 reports moments of the resulting price distribution when $N = 2$.⁵⁶ Results for Logit demand (see Section A.2) are also reported.

TABLE A2 Pricing outcomes in Cournot.

Algorithm	N	Minimum	25th Percentile	Median	75th Percentile	Max
Perfect synchronous	2	4.67	4.67	4.67	4.67	4.67
Asynchronous	2	5.24	5.65	5.84	6.00	6.42

Note: For each N , 100 simulation runs were conducted. The distribution of prices once all simulations have reached a rest point is reported.

TABLE A3 Pricing: Asynchronous versus perfect synchronous.

Algorithm	N	Minimum	25th Percentile	Median	75th Percentile	Max
Homogenous Bertrand						
Perfect synchronous (Firm 1)	2	2.13	2.13	2.13	2.13	2.13
Asynchronous (Firm 2)	2	2.13	2.13	2.13	2.13	2.13
Differentiated Bertrand (Logit)						
Perfect synchronous (Firm 1)	2	2.73	2.94	3.69	4.45	7.28
Asynchronous (Firm 2)	2	2.84	3.14	4.10	4.95	7.98

Note: For each N , 100 simulation runs were conducted. The distribution of prices once all simulations have reached a rest point is reported.